Part V

3D LiDAR SLAM and Autonomous Inspection

Simultaneous Localization and Mapping (SLAM)

How to Build a Vision-Based Navigation and Localization Drone



Editor: Lee Shihyuan (李世淵)

LINE OA @prof

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Part V 3D LiDAR SLAM and Autonomous Inspection

Chapter 17

3D Point Clouds and LiDAR Mapping

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Part V begins here, and what this chapter does is lift everything accumulated in the first four parts out of the plane and into three dimensions. The change sounds like no more than adding a z coordinate, but more is altered than one would expect: the map is no longer a picture that can be printed but tens of millions of three-dimensional points; registration is no longer three degrees of freedom but six; the volume of data jumps from a few megabytes to a few gigabytes. The four tools of this chapter—downsampling, ICP, NDT, and map representation—exist precisely to cope with that jump in scale.

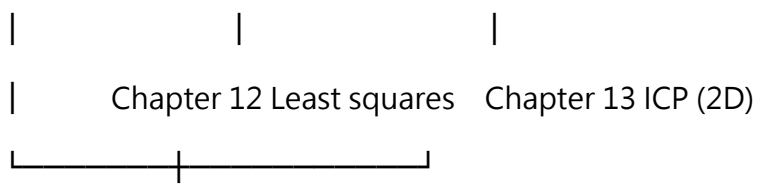
It is worth noting that this chapter contains almost no new mathematics. ICP is the three-dimensional version of Section 13.3, the core of NDT is the multivariate Gaussian distribution of Chapter 11, the optimization behind registration is the least squares of Chapter 12, and the coordinate transformation is the rigid-body motion of Chapter 10. What is genuinely new is only scale—and the engineering problems that scale brings. That is also why this chapter devotes a whole section to downsampling: in a three-dimensional world, how to make data smaller without distorting it is a subject in its own right.

Figure 17-0 The Link Map of Chapter 17

Existing tools from the first four parts

Chapter 8 3D LiDAR Chapter 10 Rigid-body transformation Chapter 11 Gaussian

distribution



Learning Objectives

The 2D SLAM of Part IV lets a robot localize and map on a plane, but the real world is three-dimensional: stairs have height, ceilings carry pipework, a legged robot must step over obstacles, and a drone must thread through three-dimensional space. This chapter raises the dimension from the plane to the solid, introducing the processing and registration of three-dimensional point clouds and opening Part V's treatment of 3D SLAM and autonomous inspection. After studying this chapter, you should be able to:

1. Describe the data structure, sources, and characteristics of a 3D point cloud, and understand how it differs fundamentally from a 2D occupancy grid map.
2. Explain the principle and the necessity of voxel downsampling, and estimate its effect.
3. Describe the procedure, the variants, and the limitations of ICP point cloud registration.

4. Explain the central idea of NDT and compare it with ICP in terms of differences and suitable applications.
5. Describe the ways a 3D map can be represented and the procedure for scanning an indoor space.
6. Explain the principle of point cloud colorization and the applications of colored three-dimensional visual modeling.
7. Give examples of the practical value of 3D spatial modeling in factories, on campuses, and at exhibition venues.

Core Concepts

中文名詞	English Term	Explanation
點雲	Point Cloud	Spatial data made up of a large number of three-dimensional points, each with x, y, and z coordinates.
體素	Voxel	A cubic cell in three-dimensional space, the counterpart in three dimensions of the 2D pixel.
體素降採樣	Voxel Grid Downsampling	Cutting space into voxels and keeping one representative point per cell to reduce the count.
點雲配準	Point Cloud Registration	Finding the rotation and translation between two point clouds that bring them into alignment.
迭代最近點	ICP	The classic registration algorithm that repeatedly finds corresponding points and solves for the best transformation.
點對面 ICP	Point-to-Plane ICP	A variant of ICP taking the point-to-plane distance as the error, which converges

中文名詞	English Term	Explanation
		faster.
常態分布轉換	NDT	A registration method that converts a point cloud into Gaussian distributions within voxels and then matches.
法向量	Normal Vector	The vector describing the orientation of the surface a point lies on, used in point-to-plane registration and in shading.
KD 樹	KD-Tree	The spatial data structure that accelerates nearest-neighbor search, the key to ICP's efficiency.
八叉樹	Octree	A tree structure representing three-dimensional space by recursive subdivision, used to compress a map.
點雲顏色化	Colorization	Projecting the color of camera imagery onto a point cloud to produce a colored model.
數位分身	Digital Twin	A high-accuracy three-dimensional digital model of a real space or piece of equipment.

17.1 Point Cloud Data

Chapter 8 introduced the point cloud as the output format of a 3D LiDAR; this section goes deeper into its structure and characteristics, which are the data foundation of all of Part V.

What Is a Point Cloud

A point cloud is a set made up of a large number of three-dimensional points. In its most basic form, each point carries only three numbers—x, y, and z coordinates—meaning "there is a surface at this position in space." Plot several hundred thousand such points and the three-dimensional shape of an object and its surroundings emerges.

Each point may also carry additional fields:

- Intensity: the strength of the laser return, which reflects the material and the lightness of the surface. Metal and retroreflective signs give a high intensity, black objects a low one.
- Color (RGB): if camera imagery has been fused in, each point can carry a color, forming a colored point cloud (Section 17.6).
- Ring: in a multi-beam LiDAR, which laser line the point came from. This information is very important in the LOAM feature extraction of Chapter 18.
- Timestamp: the precise instant at which the point was measured, used for motion compensation (Chapter 18).

Three Characteristics of a Point Cloud

- Unordered: a point cloud is a set of points with no fixed ordering. This is unlike the regular pixel grid of an image—convolution can be applied directly to an

image but not to a point cloud. It is also one of the great difficulties of processing point clouds with deep learning.

- Sparse and uneven: points are dense nearby and sparse far away (the angular resolution formula of Chapter 8); and only surfaces carry points, with the interior of objects and anything not scanned left empty.
- Enormous in volume: a 64-beam LiDAR can produce more than a million points per second. A ten-minute scan is several hundred million points—which is why Section 17.2 must downsample.

The Fundamental Difference from a 2D Occupancy Grid Map

Characteristic	2D Occupancy Grid Map (Chapter 13)	3D Point Cloud
Data structure	A regular grid with one probability per cell	An unordered set of points
Spatial representation	The three states of occupied, free, and unknown	Only "a surface is here"
Unknown region	Marked explicitly (in gray)	Cannot tell "nothing there" from "not scanned"
Memory	Proportional to area	Proportional to point count, usually far larger
Usable directly for planning	Yes	Not conveniently; conversion is needed
Fidelity of detail	Limited by resolution	Very high; the raw measurements are preserved

Table 17-1 2D Occupancy Grid Map Compared with a 3D Point Cloud

Pay particular attention to the row for the unknown region. Chapter 13 stressed

that an occupancy grid map can tell "definitely empty" from "unknown," which is crucial for path planning; but a bare point cloud has no such ability—a place with no points may be truly empty or may simply never have been scanned. This is exactly why point clouds are so often converted into a 3D occupancy grid (an octree, say) in practice (Section 17.5).

17.2 Voxel Grid Downsampling

Faced with data that routinely runs to tens of millions of points, the first thing to do is almost always to downsample—to reduce the number of points while preserving the shape as far as possible. Voxel downsampling is the most common and the most intuitive method.

The Principle: One Representative per Cell

A voxel is the three-dimensional counterpart of a pixel—space cut into small cubes of fixed side length. The procedure of voxel downsampling is extremely simple:

1. Cut the space occupied by the whole point cloud into a voxel grid of side length d.
2. Assign each point to the voxel it falls in.
3. Output one representative point per voxel—usually the centroid (the mean coordinate) of all points in that voxel.
4. Ignore empty voxels entirely.

The result is that the many points of an originally dense region are merged into a few, while sparse regions are barely affected. The point cloud as a whole becomes uniform and light, yet the shape is largely preserved.

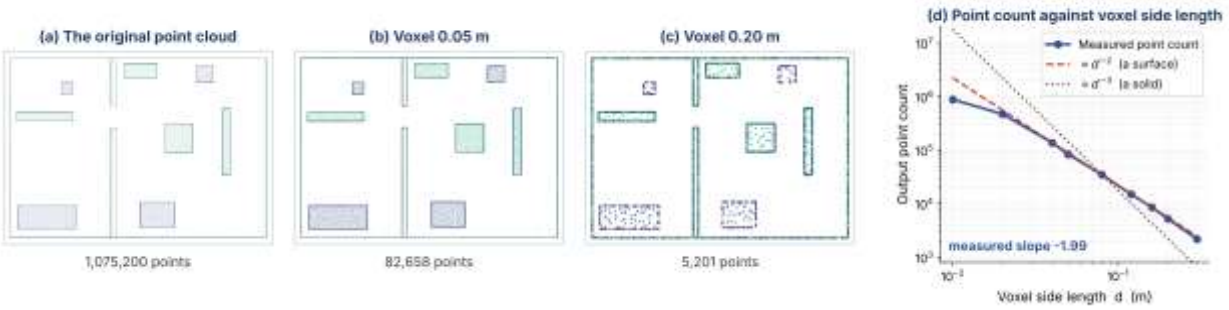
Centroid or Cell Center

Here lies an implementation difference that looks minor but matters. Taking the centroid (the center of mass) of all points in a voxel reflects the true surface position more faithfully; taking the voxel center for speed is faster but less accurate. Comparative studies show that exact voxel downsampling gives results all but identical to the standard implementation, while the approximate version may produce spurious points—as many as twice the point count. For mapping where accuracy matters, the exact centroid version is recommended.

The Effect and the Price of Downsampling

- Effect: the larger the voxel side length d, the fewer points are kept. The reduction in point count is roughly inversely proportional to the cube of d—double the side length and the count falls to about an eighth. This is the inevitable consequence of three-dimensional space.
- Price: loss of detail. If the voxel side length exceeds the characteristic size of an object (a 20-centimeter voxel against a 5-centimeter pipe, say), that feature disappears.
- Trade-off: d should be slightly smaller than the smallest feature size you care about. Indoor mapping commonly uses 5 to 10 centimeters; large outdoor areas can use 20 to 50 centimeters.

Figure 17-1 Voxel Downsampling: A Visible Trade-off and a Law You Can Check



This figure comes from a genuine simulated scan: an indoor space of 10 × 7 × 2.8 meters, scanned from seven positions by a 64-beam LiDAR and merged, for 1,075,200 points in all. Panels (a) to (c) are top views (the floor and ceiling are filtered out of the display so the structure can be seen clearly), with color representing height. The walls, the central partition and the doorway, and the outlines of the bookcase, the workbench, and the equipment rack are all visible.

Comparing the three panels makes the trade-off of downsampling clear. At a voxel side length of 0.05 meters (panel b), the point count falls from 1.075 million to 82,658, a reduction of more than ninety percent, yet the wall lines, the furniture outlines, and the position of the doorway are all preserved—this is what Section 17.2 means by "the point cloud as a whole becomes uniform and light, yet the shape is largely preserved." At 0.20 meters (panel c), only 5,200 points remain, the walls begin to break up into strings of disconnected points, the furniture degenerates into a few patches of color, and the doorway is already hard to make out. The three pipes of 18 centimeters' diameter on the ceiling of this chapter are close to vanishing at this scale as well.

Panel (d) turns all this into a law you can check by arithmetic. The horizontal axis is the voxel side length and the vertical axis the output point count, both on logarithmic scales. The measured points fall on a straight line of slope −1.99—as near as makes no

difference to the −2 predicted by Equation 1, and clearly away from the −3 of a solid volume (the dotted line in the figure). This confirms the easily overlooked point of Section 17.2: what the LiDAR scans is the surface of an object rather than its interior, so a point cloud is essentially a two-dimensional surface embedded in three-dimensional space, and the point count falls with the square rather than the cube of the side length. The practical rule of thumb is simply this: double the voxel side length and the point count falls to about a quarter.

The only departure from the straight line is the leftmost point, at 0.01 meters. The reason is not hard to see: once the voxel is smaller than the average spacing of the original points, most voxels contain only one point, downsampling does almost nothing, and the count is pinned by the original sampling density. This gives a useful test—if you shrink the voxel and find the point count no longer rising appreciably, you have reached the resolution floor of the sensor itself, and going smaller only wastes computation.

How to Choose the Voxel Side Length

The rule: d should be slightly smaller than the smallest feature size you care about. To see a 5-centimeter pipe, the voxel cannot be 10 centimeters.

Indoor mapping commonly uses 0.05 to 0.10 meters; large outdoor areas or vehicle-mounted work may use 0.20 to 0.50 meters.

The point cloud used for registration may be sparser than the one used for display—the registration experiment of Figure 17-3 in this chapter was run at 0.06 meters.

The check: shrink the voxel step by step and record the point count. When the

count no longer rises with it, the resolution limit of the sensor has been reached.

Recent systems such as KISS-ICP have one detail worth noting: they keep one original point in each voxel rather than the centroid, to avoid the discretization error that computing a centroid introduces (see Section 17.8).

Other Downsampling Strategies

Besides the voxel method, several other strategies are common, each with its own place:

- Random downsampling: discard a fixed proportion of the points at random. The fastest, but it can leave the distribution uneven.
- Uniform downsampling: take one point every fixed interval; simple, but it ignores density.
- Distance threshold: guarantee that output points are at least a given distance apart.
- Farthest point sampling: repeatedly choose the point farthest from those already chosen; the most evenly distributed but slower to compute, and common as preprocessing for deep learning.
- Edge-preserving downsampling: a recent line of improvement that adjusts the downsampling density according to point cloud features and deliberately retains edge information, so that key features are not lost.

The implementations in this book use voxel downsampling throughout—it strikes the best balance among effect, speed, and implementation difficulty, and it is the default choice of mainstream libraries such as PCL and Open3D.

17.3 ICP Point Cloud Registration

Point cloud registration is the central computation of 3D SLAM: given two partly overlapping point clouds, find the rotation and translation needed to align one with the other. This is precisely the three-dimensional version of the scan matching of Section 13.3, and ICP remains the classic method.

The Three-Step Loop of ICP

The procedure of Iterative Closest Point (ICP) is exactly the same as the 2D version introduced in Chapter 13, only carried out in three-dimensional space:

5. Find correspondences: for each point in the source cloud, find the nearest point in the target cloud and pair them.
6. Solve for the transformation: compute a rotation R and a translation t (the rigid-body transformation of Chapter 10) that minimize the sum of squared distances between all paired points.
7. Update and iterate: apply the transformation to the source cloud, return to step 1 and find correspondences afresh, repeating until the change in the transformation falls below a threshold (convergence).

The Key to Efficiency: the KD-Tree

Finding the nearest point in step 1 is the most time-consuming part of ICP. With brute force, every point must be compared against every point of the target cloud, which is entirely impractical for a cloud of several hundred thousand points.

The remedy is a KD-tree—a data structure that recursively partitions space and can reduce nearest-neighbor search from linear to logarithmic time. In practice the efficiency of building and searching the KD-tree often determines the speed of ICP as

a whole. It is also why modern registration libraries place such emphasis on parallelized KD-tree implementations.

Important Variants of ICP

- Point-to-point ICP: the original form, in which the error is the straight-line distance between two points. Simple to implement, but slow to converge when sliding along a plane.
- Point-to-plane ICP: the error becomes the perpendicular distance from the source point to the plane containing the target point. This change lets points slide freely along a plane without penalty and greatly accelerates convergence—it is the version most used in practice. The price is that a normal vector must first be estimated for every point.
- Generalized ICP (GICP): considers the local planar structure of both source and target, and is more robust still.
- Voxelized accelerated versions: downsample before registering, or use a voxel structure to accelerate the correspondence search, raising speed while holding accuracy.

Three Limitations of ICP

- It needs a good initial guess: ICP is a local optimization, and if the initial positions of the two clouds are too far apart it converges to a wrong local optimum. This is the old problem already stressed in Section 13.3—in practice odometry or an IMU supplies the initial value.
- It fears structural degeneracy: in a long corridor, a tunnel, a large flat surface, or any scene short of geometric constraints, the point cloud can slide freely in some direction with no change in error, and ICP cannot determine the

displacement along it. This is the three-dimensional version of the corridor predicament of Section 13.6.

- It is sensitive to outliers: dynamic objects (people walking by) and noisy points drag the registration result around. In practice it must be paired with outlier rejection or a robust kernel (Chapters 12 and 15).

17.4 NDT: the Normal Distributions Transform

ICP has one fundamental bottleneck in efficiency: every iteration must find nearest-point correspondences. The Normal Distributions Transform (NDT) proposes an entirely different line of attack—do not look for corresponding points at all, but turn the point cloud into a continuous probability distribution.

The Central Idea: Turn the Point Cloud into a Probability Field

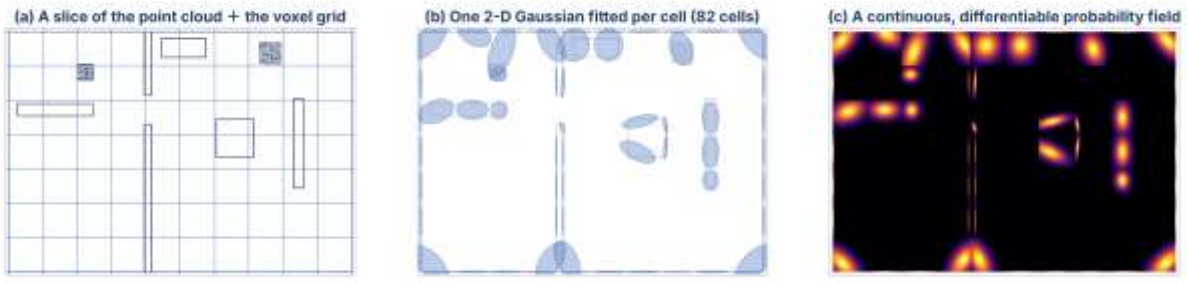
What NDT does is this:

8. Cut the space occupied by the target point cloud into voxels (cells).
9. For each voxel, compute the mean and the covariance of all points inside it—that is, use a three-dimensional Gaussian distribution to describe roughly what the points of that cell look like (the Gaussian distribution of Chapter 11).
10. The whole point cloud thereby becomes a continuous probability field pieced together out of many Gaussian distributions: at any position in space one can compute the probability density that a surface is there.
11. To register, find the transformation that maximizes the total probability of all points of the source cloud within this probability field.

This transformation brings one decisive advantage: the objective function becomes a smooth, differentiable, continuous function that can be solved directly by

optimization methods such as Newton's method (Chapter 12), and there is no need whatever to search repeatedly for nearest points during the iterations.

Figure 17-2 The Core of NDT: Turning Discrete Points into a Continuous Probability Field



"Turn the point cloud into a probability field" sounds abstract, but drawn out it is in fact very intuitive. For clarity this figure takes a horizontal slice of the indoor space of the previous figure at a height of 1.0 meters, reducing the problem to two dimensions for the demonstration; the three-dimensional procedure is exactly the same, only with the Gaussian distributions becoming ellipsoids instead of ellipses.

Panel (a) is the sliced point cloud with a voxel grid of side length 1.0 meters laid over it. At this stage the data is still discrete points, and to ask whether there is a surface at some position in space one can only search for nearby points—exactly what ICP must do at every iteration, and the most time-consuming thing it does. Panel (b) then computes the mean and covariance of the points in each cell and describes them with a two-dimensional Gaussian distribution. Note that the shape of each ellipse itself carries information: in a cell lying on a wall, the ellipse is drawn out into a long thin sliver whose major axis runs along the wall; in a cell at a corner, the ellipse is closer to circular. In other words, the covariance matrix automatically records which way the surface in that cell extends.

Panel (c) pieces these 82 Gaussian distributions together into a continuous probability field. Now any point in space can be substituted straight into a formula to obtain the probability density that a surface is there, with no search and no KD-tree. More important still, this function is smooth and differentiable, so it can be handed directly to the Newton's method of Chapter 12 to find an extremum. The two-step find-correspondences-then-solve loop of ICP is compressed in NDT into a single optimization problem.

The figure also honestly exposes one weakness of NDT: look closely at panel (c) and the probability field is seen to break at the voxel boundaries, so that where one wall crosses two cells a seam appears in the bright band. This is because each cell computes its own Gaussian and nothing is continuous from cell to cell. Real NDT implementations use overlapping voxels or interpolation to soften the problem, and this is precisely why the voxel resolution discussed in the next section is NDT's most sensitive parameter.

Reading the Ellipse: the Geometric Meaning of the Covariance

The direction of the ellipse's major axis = the direction of greatest variance in the cell's point distribution, which is the direction in which the surface extends.

The flatter the ellipse = the more the cell's points concentrate on a single plane, and the stronger the constraint in the direction of the normal.

Close to circular = the point distribution has no marked directionality (a corner, a cluttered area, or too few points).

The implementation in this book sets a lower bound of 0.05 meters on the covariance: a real surface carries measurement noise and should not be described as having zero thickness. Without this regularization the Gaussian

degenerates into a line and the optimization becomes extremely unstable.

A cell must hold at least five points before a Gaussian is built—with too few points the covariance estimate is unreliable, and it is better to leave the cell blank.

NDT Compared with ICP

Item of Comparison	ICP	NDT
Core computation	Finding nearest-point correspondences	Gaussian distributions within voxels
Does it search for correspondences	Yes (accelerated by a KD-tree)	No
Speed on large point clouds	Slower	Faster
Accuracy	Usually higher	Slightly below ICP
Sensitivity to the initial value	More sensitive	Less sensitive, with a wider basin of convergence
Key parameter	Correspondence distance threshold	Voxel resolution
Suitable scenes	Fine registration with fewer points	Large areas, large point clouds, a poor initial value

Table 17-2 ICP Compared with NDT

Comparative studies report that because NDT does not compute with the features of corresponding points during registration, it is faster than ICP on large point clouds

but less accurate. This conclusion appears quite consistently across a number of experiments.

Figure 17-3 ICP against NDT Head to Head, and Structural Degeneracy in a Long Corridor

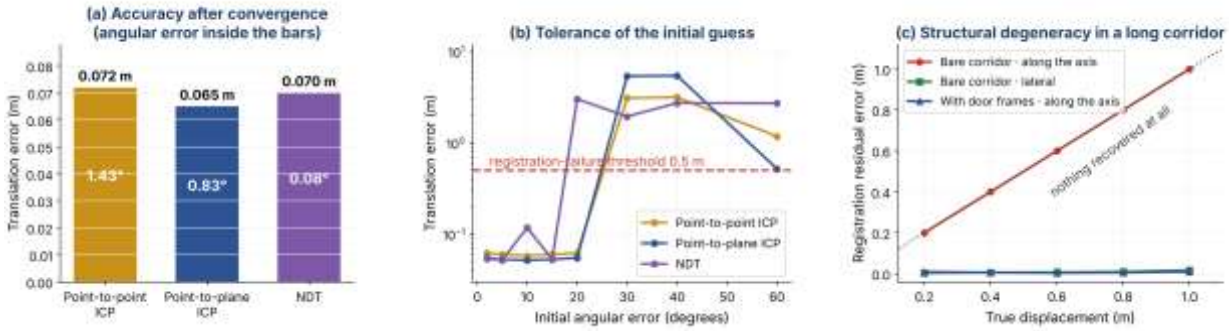


Table 17-2 compared the two methods in words; this figure turns that into measurement. The setup is a genuine SLAM situation: a LiDAR scans the same indoor space once from each of two positions (about 3.9 meters apart and rotated by 34 degrees), the two clouds overlap only partly, and each is voxel downsampled at 0.06 meters to about twenty-three thousand points. The task is to align the second back onto the first, and it is deliberately given an inaccurate initial guess.

Panel (a) is the accuracy after convergence. The translation error of all three methods falls between 6.5 and 7.2 centimeters, a narrow spread—this figure is in fact the floor imposed by the two viewpoints sampling the surface differently, not the limit of the algorithms. What really separates them is the angular error (the numbers inside the bars): 1.43 degrees for point-to-point ICP, 0.83 degrees for point-to-plane ICP, and 0.08 degrees for NDT. Point-to-plane is clearly better than point-to-point, which bears out the argument of Section 17.3 that points can slide freely along a plane without penalty; and NDT, because the covariance itself encodes the direction of the surface, does best of all on angle.

Panel (b) is tolerance of the initial guess, and it is the most practical panel in the figure. The horizontal axis is the initial angular error and the vertical axis the final translation error (on a logarithmic scale). All three methods converge stably to the centimeter level within 15 degrees; NDT begins to fail at 20 degrees, while both forms of ICP hold at 20 degrees and collapse at 30. The shape of the curves deserves notice: it is not gradual degradation but failure off a cliff—once outside the basin of attraction the result is not worse but wholly wrong, with the error jumping straight to the meter level. This explains why odometry or an IMU must supply the initial guess in practice (the first limitation of Section 17.3): they need not be very accurate, only accurate enough to hold the error within a dozen or so degrees.

Panel (c) then examines the second limitation of Section 17.3 on its own: structural degeneracy. The scene becomes a featureless corridor 90 meters long and 3.2 meters wide, far longer than the LiDAR's 25-meter range, so that the robot never sees either end. The red line is motion along the corridor's axis: the residual error equals the true displacement itself—displace by 1.0 meter and the error is 1.0 meter, meaning that registration has recovered no displacement whatever. The green line is lateral motion, with an error of only a centimeter or two, entirely normal. The blue line is the axial test repeated after doorways and columns are added along the same corridor: the error falls at once to the centimeter level.

These three lines could hardly make structural degeneracy clearer: the problem lies not in the algorithm but in the geometric information itself. After a translation along the axis of a featureless corridor the point cloud looks exactly the same, and the registration algorithm has nothing on which to judge how far it has moved—no optimizer, however good, can conjure information that does not exist. Only two

remedies exist: use another sensor (wheel odometry, an IMU, UWB) to supply the constraint in that direction, or make sure the environment holds enough geometric features. This also answers the tunnel difficulty of Case 17-2.

An Honest Note About This Comparative Experiment

Speed is not included in the comparison. The NDT here is a teaching implementation written in Python while the ICP is Open3D's C++ implementation, and with an order of magnitude between them in engineering optimization, comparing run times would not be fair. The conclusion in the literature that NDT is faster (Section 17.4) comes from implementations optimized to the same degree.

The tolerance result differs from the common claim that NDT is less sensitive to the initial value. In this experiment ICP uses a coarse-to-fine correspondence distance schedule (1.5 → 0.75 → 0.3 meters), which favors ICP considerably; NDT's basin of attraction depends on the voxel size, and this book tried a number of schedules from 2 meters to 8 meters, of which starting at 2 meters was best.

How the conclusion should be read: all three methods work given a reasonable initial value, and point-to-plane ICP and NDT beat point-to-point on angular accuracy; while the two limitations that the initial value must be good enough and that a degenerate scene has no solution apply to all three alike.

You can reproduce these three panels in Parts Two and Three of Exercise 17-1; different numbers are normal, and what matters is the trend.

NDT's Key Parameter: Voxel Resolution

The most important parameter of NDT is the side length of the voxel (cell). Too

small and there are too few points per cell for a reliable Gaussian estimate; too large and the description of space is too coarse, so accuracy falls. One study measuring point clouds in a tunnel environment found that a voxel resolution of 0.5 meters met the resolution requirement with the shortest registration time. This is a reminder that the best parameter is closely tied to the scale of the scene—indoors, in a tunnel, and outdoors all differ, and the value must be measured and tuned.

NDT and ICP in Cooperation

Since NDT is fast and robust while ICP is accurate, the natural thought is to combine them: use NDT for a coarse registration that quickly closes the gap, then ICP for a fine registration that raises the accuracy. This pairing of coarse NDT with fine ICP is quite common in practical systems, and it again embodies the idea this book stresses repeatedly: there is no universal algorithm, only an appropriate combination.

17.5 3D Maps and Indoor Space Scanning

With downsampling and registration in hand, a succession of LiDAR scans can be stitched into a complete three-dimensional map. This section explains how a 3D map may be represented and the practical procedure for scanning indoors.

Three Representations of a 3D Map

- Raw point cloud map: store all the (downsampled) points directly. Highest in fidelity and best-looking, but large, unable to tell empty from unknown, and awkward for planning paths directly. Suited to surveying and to display.
- 3D occupancy grid (octree): cut space into voxels and record an occupancy probability, subdividing recursively with an octree—subdividing only where something is present and representing open space with large cells, which saves

a great deal of memory. This representation can distinguish the three states of occupied, free, and unknown, and suits robot navigation and obstacle avoidance. OctoMap, mentioned in Chapter 3, is the representative example.

- Mesh model: convert the point cloud into a surface model of triangular facets. Small in file size, able to carry textures, and suited to architectural and game applications, but the reconstruction is more complex and may distort the result.

The Procedure for Scanning an Indoor Space

12. Plan the scanning route: make sure all the space you want is covered, and leave enough overlap at the critical places (corners, between rooms), since insufficient overlap causes registration to fail.
13. Collect the data: scan slowly, moving with a backpack, a handheld unit, or a robot platform (Section 15.6). Move slowly and avoid violent shaking, which causes motion distortion.
14. Preprocess: remove outliers, crop the extent as needed, and voxel downsample (Section 17.2).
15. Register and stitch: align neighboring scans with NDT or ICP (Sections 17.3 and 17.4), together with the pose estimates from SLAM.
16. Optimize globally: detect loop closures and run graph optimization (Chapters 12 and 15) to remove accumulated drift and secure overall consistency.
17. Post-process and output: convert to an octree or a mesh or add color as needed (Section 17.6), and output in a standard format (PLY, PCD, or LAS).

Quality Assessment

Echoing the assessment of 2D maps in Section 13.7, a 3D map must also be checked: whether the walls have grown thick (a sign of registration error, with one wall

drawn as two layers), whether straight structures are straight, whether the loop closure lines up, and how measured key dimensions compare with surveyed values. Thickened walls are the most typical quality indicator in 3D mapping—an ideal wall should be a thin single layer of points.

17.6 Point Cloud Colorization and Colored Visual Modeling

A purely geometric point cloud is accurate but looks like a gray-white skeleton, hard to interpret and unattractive. Only with color added does a point cloud truly become a digital twin.

Where the Color Comes From: the Projection Principle

The color comes from camera imagery. The principle is the pinhole projection model of Chapter 9 used in reverse:

18. The camera's intrinsic parameters K (Chapter 9) and the extrinsic parameters of the camera relative to the LiDAR (the coordinate transformation of Chapter 10) are known.
19. For each three-dimensional point, first use the extrinsics to take it from LiDAR coordinates into camera coordinates.
20. Then use the intrinsics to project it onto the image plane, obtaining the corresponding pixel coordinates.
21. Read the RGB value of that pixel and assign it to this three-dimensional point.

This procedure again confirms the central value of Chapter 10: accurate coordinate transformation and extrinsic calibration are the foundation of multi-sensor fusion. Let the extrinsics be off by a few degrees or a few centimeters and the color is pasted crooked—the color of the wall ends up on the window.

Practical Challenges

- Time synchronization: LiDAR and camera timestamps must be aligned. In motion, a discrepancy of a few tens of milliseconds misplaces the color (the synchronization problem of Chapters 5 and 10).
- Occlusion handling: a point the camera cannot see (blocked by something in front) should not be given the color of the thing in front. A visibility test is needed.
- Exposure and color difference: images taken at different moments differ in brightness, so patches of color appear in the stitched cloud. Color balancing is needed.
- Mismatched fields of view: a LiDAR is often 360 degrees while a camera's field of view is limited. Several cameras are needed, or one accepts that some points have no color.

The Value of a Colored Point Cloud

- Readability: a colored model can be understood at a glance even by non-technical people—which matters enormously when presenting project results to a client or to the university.
- Semantic cues: color provides clues to material and object, an important input to later AI semantic segmentation (Chapter 32).
- Preservation of cultural assets: in the digital archiving of monuments and artifacts, color and texture are indispensable information.
- Defect detection: wall cracks, corrosion, and water stains are not obvious geometrically but are plain to see in color.

17.7 3D Spatial Modeling of Factories, Campuses, and Exhibition Venues

This section explains the practical value of 3D mapping, with particular attention to directions that can be carried out on a university campus.

Factories: Facility Management and Digital Twins

A three-dimensional model of a factory can serve production line layout planning (will the new machine fit? will the transport path collide with anything?), the recording of piping and fire safety equipment, and spatial queries for equipment maintenance. Applications of this kind combine a preloaded floor plan outline to plan a coverage path that reaches every accessible room, corridor, and machine space, while keeping the robot safely clear of people and equipment during the building's operating hours.

The key advantage of LiDAR over a camera is especially plain here: unlike camera-based methods, which perform poorly in low light or in textureless areas (machine rooms and parking structures, say), LiDAR works reliably whatever the lighting, which makes it the clear choice for facility maintenance applications.

Campuses: Directions for Implementation on a University Site

- Architectural surveying: scan the interior of a department building to produce a three-dimensional model usable for teaching, space management, and accessibility planning.
- Indoor navigation maps: build the spatial foundation for the semantic navigation of Part IX, so that the robot knows where the elevator is and where the stairs are.

- A digital twin of the laboratory: scan the intelligent robotics classroom as the basis of a simulation environment—which lets students test algorithms in a virtual classroom and then move to the physical one for verification.
- Teaching material: the scan itself is excellent teaching material, letting later students see a three-dimensional model of what our own school looks like.

Exhibition Venues and Cultural Assets

A three-dimensional model of an exhibition venue can serve exhibit planning, virtual visits, and circulation analysis; and the digital preservation of monuments and historic buildings is one of the socially most valuable applications of 3D scanning—once a building is damaged by disaster or by time, an accurate three-dimensional record becomes the only basis for restoration. Taiwan has an abundance of historic buildings, which makes this a project direction of both technical and humanistic significance.

The Technical Foundation: Edge Computing

It is worth noting that modern 3D mapping leans more and more on edge computing platforms. SLAM algorithms run on edge processors such as the NVIDIA Jetson, stitching individual scan frames together and correcting positional drift through loop closure—the technique by which a robot recognizes an area it has visited before and uses that recognition to correct accumulated error in its trajectory estimate. Recent years have also brought GPU-accelerated voxel reconstruction libraries that can build signed distance fields and occupancy grids in real time, letting a robot reconstruct the geometry of a facility while it navigates, supporting mapping and obstacle avoidance at once.

This means 3D SLAM is no longer a technology that demands a workstation; a

handheld Jetson is equal to it—excellent news for teaching and student projects, and the direction pursued in depth in Part VII (edge AI SLAM) and Chapter 32.

17.8 Where Point Cloud Processing Is Going: Four Developments of 2024–2026

The original ICP paper was published in 1992 and NDT in 2003—by the standards of the techniques in this chapter, they are thirty- and twenty-year-old methods. Interestingly, far from being superseded, they have lately been the subject of a return to basics. This section gathers four developments worth noting.

1. A Return to Simplicity: KISS-ICP's "In Defense of Point-to-Point ICP"

KISS-ICP, published by a team at the University of Bonn in 2023, carries the subtitle "In Defense of Point-to-Point ICP" outright. Their claim is quite provocative: rather than piling complexity onto LiDAR odometry—feature extraction, deep learning modules, loop closure detection, pose graph optimization—get a few parts of the most basic point-to-point ICP right and the result can stand alongside the state of the art.

The parts they got right number four: predict motion with a constant-velocity model and compensate scan distortion with it (needing neither IMU nor wheel encoder), build the local map by voxel downsampling, optimize with a robust kernel, and, most crucially, use an adaptive correspondence distance threshold—a threshold not set by hand but computed automatically from a statistic of how far the odometry actually departs from the motion prediction (three times the standard deviation). The result is a system that needs almost no tuning, with one set of settings carrying across vehicle-mounted, aerial, and handheld platforms.

For students taking this course the lesson of this case matters more than its technical

detail: when your registration works poorly, the first response should not be to reach for a more complex algorithm but to check the basics of downsampling, initial guess, and correspondence distance threshold. The cliff-edge curve of Figure 17-3(b) in this chapter has already shown that the initial value and the threshold matter far more than the choice of algorithm.

2. Engineering Efficiency: from Comparing Algorithms to Comparing Implementations

The second development is that the fact that one algorithm can differ several-fold between implementations has formally been put on the table. The representative work is the small_gicp library published in 2024 by Kenji Koide of Japan's National Institute of Advanced Industrial Science and Technology: it rewrites and parallelizes downsampling, KD-tree construction, and registration throughout, and reports voxel downsampling about 1.3 times faster than PCL single-threaded and 3.2 times faster with six threads, KD-tree construction up to 6 times faster than nanoflann, and generalized ICP about 2.4 times faster than PCL's version on a single thread alone.

This echoes the discussion of tool choice in Case 17-3 and adds a dimension this chapter had not yet mentioned: when you write in a project report that "we used ICP for registration," that sentence has in fact said nothing definite about performance—which implementation, whether multithreading was enabled, and how the KD-tree was built may affect the speed you actually see more than the difference between point-to-point and point-to-plane.

3. A New Option for Map Representation: 3D Gaussian Splatting

The third development strikes at the list of three 3D map representations in Section

17.5. Over the last two years, 3D Gaussian Splatting has rapidly become a fourth option: it represents a scene with a great many three-dimensional Gaussians carrying position, shape, color, and transparency, so that it can be fitted directly from data like a point cloud and yet rendered into photorealistic images like a mesh.

This line has already reached the LiDAR field. Splat-LOAM in 2025 takes Gaussian splatting directly as the representation for LiDAR odometry and mapping, while work such as LiV-GS integrates LiDAR and camera into one Gaussian splatting SLAM for outdoor scenes. Students may notice an interesting coincidence: NDT describes a surface with a Gaussian inside a voxel (Figure 17-2 of this chapter), and Gaussian splatting describes the appearance of a scene with Gaussians in space—twenty years apart, both arrived at the same answer of describing the world with a collection of Gaussian distributions.

4. The Division of Labor between Learned Registration and Traditional Methods

The last development is that the place of deep learning in the registration problem has gradually become clear. The strength of learned methods lies in the step of finding correspondences—a learned three-dimensional descriptor can find the correspondence between two clouds outright even with very little overlap and a very poor initial guess, exactly where ICP and NDT are weakest (the cliff in Figure 17-3(b)). But once the correspondences are found, solving for the transformation remains the least squares problem of Chapter 12, at which traditional methods are both fast and accurate.

The practical division of labor at present is therefore quite clear: global registration (with no initial value, as in relocalization and loop closure) tends to use learned

descriptors or random sample consensus (RANSAC); local refinement (where a decent initial value already exists, as between consecutive frames) remains the domain of ICP and NDT. And the existence of KISS-ICP reminds us that in the particular task of LiDAR odometry over consecutive scans the initial value has always been good enough, which is why traditional methods have to this day had no reason to be replaced.

The Common Thread of the Four Developments

Method (KISS-ICP): complexity does not equal effect, and getting the basics right is more useful than changing algorithms.

Engineering (small_gicp): different implementations of one algorithm can differ several-fold, and performance is a question of implementation and not only of theory.

Representation (Gaussian splatting): map representation expands from points, voxels, and meshes to a collection of Gaussians, echoing NDT's idea across the years.

Learning (descriptors): what deep learning supplies is finding correspondences, not solving for the transformation; the two divide the labor rather than one replacing the other.

The common thread: not one of this chapter's four tools has been retired, and what has changed is how they are used and the quality with which they are engineered. Build the foundation solidly and every later change becomes readable.

Carrying Forward

This chapter has assembled the tools for processing three-dimensional point clouds:

downsampling brings the data volume back within reach, ICP and NDT let neighboring scans be aligned, and map representation and colorization let the result be used and displayed. The three sets of experiments in Figures 17-1 through 17-3 verify the law, the principle, and the limits of these three things respectively—especially the red line of Figure 17-3(c), which reminds us that some errors are not the algorithm's fault but the absence of geometric information itself.

But register neighboring scans frame by frame and accumulate, and the error still grows with distance—this is the three-dimensional version of the drift problem of Chapter 7. The LOAM family of the next chapter gives an elegant answer: rather than registering the whole cloud, first extract two classes of feature from the scan, edge points and planar points, and register only the features. This idea makes real-time three-dimensional LiDAR odometry possible, and it opens the true subject of Part V.

Formulas and Procedures

1. Estimating the Point Count of Voxel Downsampling

Apply a voxel of side length d to a point cloud and the output point count is about equal to the number of occupied voxels. Because of the nature of three-dimensional space:

$$\textit{output point count} \propto 1 \;/\; d^2 \quad \textit{(a surface-type point cloud)}$$

Note that this is the square and not the cube—because what the LiDAR scans is a surface and not a solid volume, so the points lie mainly on a two-dimensional surface. The practical experience: double the voxel side length and the point count falls to about a quarter. Only if the point cloud filled a three-dimensional volume (as in medical imaging) would the relation be cubic.

2. The Objective Function of ICP

Point-to-point ICP seeks a rotation R and a translation t that minimize the sum of squared distances between paired points:

$$(R^*, t^*) = \operatorname{argmin} \Sigma_i \, \| \, (R \cdot p_i + t) - q_i \, \|^2$$

where p_i is a point of the source cloud and q_i its nearest point in the target cloud. Point-to-plane ICP instead takes the error as the projection along the target point's normal n_i:

$$(R^*, t^*) = \operatorname{argmin} \Sigma_i \, ((\, (R \cdot p_i + t) - q_i \,) \cdot n_i \,)^2$$

The difference is that point-to-point penalizes departure in every direction while point-to-plane penalizes only departure perpendicular to the surface, letting points slide along the plane—which is exactly why it converges faster.

3. The Probabilistic Objective of NDT

NDT describes each voxel of the target cloud with a Gaussian distribution (μ, Σ). A source point p transformed by T falls in some voxel, where its score is:

$$\text{score}(p) \propto \exp(- (\, (T \cdot p - \mu \,)^{\top} \Sigma^{-1} \, (T \cdot p - \mu \,) \,) / 2)$$

The registration objective is to maximize the total score over all points:

$$T^* = \operatorname{argmax} \Sigma_i \, \text{score}(T \cdot p_i)$$

The term $(x - \mu)^{\top} \Sigma^{-1} (x - \mu)$ in the expression is precisely the core term of the multivariate Gaussian distribution of Chapter 11 (the Mahalanobis distance). You can see that the probabilistic mathematics of Part III becomes directly a tool for 3D registration here.

4. The Complete 3D Mapping Procedure

Figure 17-4 The Complete Flow from LiDAR Scan to 3D Map

① The 3D LiDAR outputs a raw point cloud (several hundred thousand to over a million points per second)

↓

② Preprocessing: outlier rejection, range cropping, motion compensation

↓

③ Voxel downsampling: reduce the point count and even out the distribution (Section 17.2)

↓

④ Feature / normal estimation: prepare for point-to-plane ICP or later processing

↓

⑤ Registration: coarse NDT → fine ICP (Sections 17.3 and 17.4)

with odometry / IMU supplying the initial guess

↓

⑥ Accumulation and stitching: merge the new scan into the global map

↓

⑦ Loop closure detection + graph optimization: remove accumulated drift (Chapters 12 and 15)

↓

⑧ Post-processing: convert to an octree / mesh, colorize (Section 17.6), output

Cases

Case 17-1 Why Walls Grow Thick: the Quality Indicator of 3D Mapping

Students doing 3D mapping for the first time often meet a puzzling phenomenon: having scanned a room and looked down on the point cloud from above, a wall that ought to be a thin single layer has become a slab more than ten centimeters thick; sometimes two parallel wall surfaces appear with a gap between them.

The cause is direct: there is registration error. The robot scans the same wall at different times from different positions, and if the pose estimate is accurate these scans should overlap perfectly into one layer; if the pose is off, the same wall is drawn at slightly different places and thickens when stacked. Let the error grow beyond a point and an obvious double wall appears (this is the three-dimensional version of ghosting, corresponding to the 2D symptom of Section 13.7).

Wall thickness is therefore the most practical quality indicator in 3D mapping: measure the spread of a stretch of wall point cloud in the direction perpendicular to the wall, and the ideal value should approach the measurement accuracy of the LiDAR itself (a few centimeters). A thickness clearly in excess of that means something is wrong with the registration or with SLAM. The check is simple, intuitive, and needs no ground truth, and students are strongly urged to make it after every mapping run—it tells you at once whether the map is usable.

Case 17-2 From Tunnels to Mines: the Three-Dimensional Challenge of Degenerate Scenes

Section 13.6 discussed how a long corridor defeats 2D scan matching, and the problem only grows worse in three dimensions. Tunnels, underground galleries, and

long corridors share one feature: along the long axis, the geometry of the environment is almost unchanging. The point cloud can slide freely in that direction without increasing the registration error—this is called degeneracy.

Researchers have developed a number of remedies for scenes of this kind. One team working in tunnel environments adopted the combined procedure of first voxel downsampling to preserve the overall structure while reducing the point count, then solving for the coordinate transformation with NDT while optimizing the voxel resolution for the features of the environment, and finally introducing a KD-tree into ICP to accelerate the point-pair search and solving with the Gauss–Newton method; measurement showed the best results at a voxel resolution of 0.5 meters.

This case offers three lessons. First, degenerate scenes are a difficulty common to all 3D SLAM, not something you did wrong. Second, NDT and ICP each have their strengths, and using them in combination often beats using either alone. Third, parameters must be tuned to the scene; no setting holds everywhere. It also foreshadows how the LOAM family of Chapter 18 attacks this problem by means of feature extraction.

Case 17-3 Choosing Tools: PCL and Open3D

Do 3D point cloud processing and you will almost certainly use two open-source libraries: PCL (the Point Cloud Library) and Open3D. The former is an established, mainly C++ tool, extremely complete in function and standard equipment in the ROS ecosystem; the latter, developed by Intel, is known for its friendly Python interface and convenient visualization, and has become quite popular in research and teaching in recent years.

Open3D provides implementations of several advanced surface registration methods, including pairwise global registration, pairwise local refinement, and multiway registration using pose graph optimization—students should recognize the last at once as the graph optimization of Chapter 12, only applied to stitching point clouds.

On performance, each has its wins. Benchmarks comparing the registration speed of different libraries show that point-to-plane ICP is usually faster than point-to-point, and that NDT's speed is quite competitive as well; recent years have also brought lightweight registration libraries designed for parallelism, which parallelize the preprocessing steps of downsampling, KD-tree construction, and normal estimation throughout, with multithreaded versions several times faster than single-threaded ones.

The advice for readers of this book: use Open3D for teaching and rapid prototyping (quick to pick up in Python, with good visualization), use PCL when integrating into a ROS system, and consider the new generation of parallelized libraries for a project that chases performance. This follows the same line as the learning path of Chapter 8—simulate first, then cheap hardware, and pursue accuracy last.

Hands-On Activities

Exercise 17-1 Point Cloud Processing and Registration

Purpose: carry out the loading, downsampling, visualization, and registration of a point cloud with your own hands, and understand the behavioral differences between ICP and NDT and the influence of their parameters. This exercise uses Python with Open3D and can be completed without 3D LiDAR hardware.

Part One: Point Cloud Basics and Downsampling (everyone)

1. Install Open3D, load one of the built-in sample point clouds (or download an indoor scan from a public dataset), and rotate it in an interactive window to get a feel for what a three-dimensional point cloud looks like.
2. Print the basic information of the point cloud: point count, coordinate range (bounding box), and whether it carries color and normals.
3. Voxel downsampling experiment: downsample at side lengths of 0.01, 0.05, and 0.2 meters, record the output point count each time, and verify whether it follows the square relation of Equation 1.
4. Visual comparison: display the original cloud and the three downsampled results side by side, and observe at what voxel size details such as wall corners and table legs begin to disappear.
5. Estimate normals and display them as colors (color corresponding to normal direction), and observe how walls, floor, and ceiling are clearly distinguished—this is the preparation for point-to-plane ICP.

Part Two: ICP Registration (everyone)

6. Take a point cloud, copy it, and apply a known rotation and translation (20 degrees about the Z axis and a translation of 0.3 meters, say) to serve as the source cloud.
7. Register with point-to-point ICP, compare the estimated transformation matrix with the truth, compute the error, and record the iteration count and elapsed time.
8. Switch to point-to-plane ICP (normals must be estimated first), compare the differences in convergence speed and accuracy, and verify the argument of Section 17.3.

9. Initial value experiment: increase the initial offset step by step (5, 20, 60, and 120 degrees), find the critical point at which ICP begins to fail, and verify for yourself that ICP needs a good initial guess.

Part Three: NDT and Combined Registration (university level)

10. Perform the same registration with NDT, compare its speed, accuracy, and tolerance of the initial value with ICP, and fill in your own version of Table 17-2.
11. Voxel resolution experiment: vary NDT's voxel size (0.2, 0.5, and 1.0 meters), observe the effect on success rate and accuracy, and compare with the discussion in Case 17-2.
12. Implement the combined procedure of coarse NDT followed by fine ICP, and compare it with using either method alone.
13. Degenerate scene test: synthesize a corridor point cloud of your own (two parallel long walls plus a floor), test whether ICP registration along the corridor loses accuracy, and reproduce the phenomenon of structural degeneracy.

Discussion Questions

1. Increasing the voxel side length from 0.01 to 0.2 meters, by what factor does the point count fall? Does it follow the square relation or the cube relation? Why?
2. How much faster is point-to-plane ICP than point-to-point? Explain the difference from the principles of Section 17.3.
3. At what initial offset does ICP begin to fail? What does this mean for a real robot system (hint: the role of odometry and the IMU)?
4. In the corridor test, in which direction does the registration lose accuracy? Is this the same problem as the 2D corridor predicament of Section 13.6?

5. (University level) If you had to choose a registration method for a campus inspection robot dog, would you choose ICP, NDT, or a combination of the two? Give your reasons and your considerations about the site.

Extended Challenge (university level)

First: download a public 3D LiDAR dataset (a sequence from KITTI, say) and implement a simple LiDAR odometry—register consecutive pairs of frames and accumulate the poses, then compare the estimated trajectory with the truth and observe how drift accumulates with distance (exactly the problem Chapter 18's LOAM sets out to solve). Second: if your school has a 3D LiDAR, actually scan a laboratory, carry out the full procedure of Figure 17-4, output a colored point cloud model, and then assess its quality by the wall thickness method of Case 17-1. This result can serve directly as the basis of Project Two in Chapter 31.

Further Reading

1. P. Besl and N. McKay, "A Method for Registration of 3-D Shapes," IEEE TPAMI, 1992. (The original ICP paper, the theoretical source shared by Chapters 13 and 17.)
2. P. Biber and W. Straßer, "The Normal Distributions Transform: A New Approach to Laser Scan Matching," IROS, 2003. (The original NDT paper.)
3. Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A Modern Library for 3D Data Processing," arXiv, 2018. (The paper introducing Open3D, with examples of registration workflows.)
4. Point Cloud Library (PCL) official tutorials (pointclouds.org). (Complete examples of downsampling, normal estimation, and registration.)

5. Open3D official documentation and Python tutorials (open3d.org). (The main tool for this chapter's exercises, friendly for teaching.)
6. A. Hornung et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," Autonomous Robots, 2013. (The representative work on 3D occupancy grid maps, echoing Section 17.5.)
7. X. Li et al., "A Tutorial Review on Point Cloud Registrations: Principle, Classification, Comparison, and Technology Challenges," Mathematical Problems in Engineering, 2021. (A complete review of point cloud registration, with a comparison of downsampling strategies.)

Summary

1. A 3D point cloud is a set made up of a large number of three-dimensional points, which may carry intensity, color, ring number, and timestamp; it has the three characteristics of being unordered, sparse and uneven, and enormous in volume, and unlike an occupancy grid map it cannot distinguish definitely empty from unknown.
2. Voxel downsampling cuts space into cubic cells and takes the centroid of each as its representative point, which greatly reduces the count and evens out the distribution; the centroid is more accurate than the cell center, and the approximate version may produce spurious points. The voxel side length should be slightly smaller than the smallest feature size of interest.
3. ICP registers point clouds by finding correspondences, solving for a transformation, and iterating, accelerating the nearest-neighbor search with a KD-tree; point-to-plane ICP converges faster because it penalizes only departure perpendicular to the surface, and it is the first choice in practice. Its

three limitations are the need for a good initial value, a fear of structural degeneracy, and sensitivity to outliers.

4. NDT turns the target point cloud into Gaussian distributions within voxels, forming a continuous differentiable probability field that needs no search for corresponding points; it is faster than ICP and less sensitive to the initial value but slightly less accurate, and its key parameter is the voxel resolution, which must be tuned to the scale of the scene.
5. A 3D map has three representations: the raw point cloud (high in fidelity), the octree occupancy grid (able to distinguish three states, suited to navigation), and the mesh model (small in file size, suited to display); the indoor scanning procedure is to plan the route, collect, preprocess, register, optimize globally, and post-process and output.
6. Point cloud colorization uses the camera projection model of Chapter 9 to assign image RGB values to three-dimensional points, the key being accurate intrinsics and extrinsics and time synchronization; a colored point cloud improves readability, provides semantic cues, and serves the preservation of cultural assets and the detection of defects.
7. 3D modeling is applied to factory digital twins, campus surveying and navigation maps, and exhibition venues and the preservation of monuments; LiDAR's indifference to illumination makes it superior to a camera at low-light and textureless sites, and advances in edge computing platforms have already taken 3D SLAM off the workstation.

This chapter has assembled the tools for processing three-dimensional point clouds: downsampling, registration, map representation, and colorization. But register

neighboring scans frame by frame and accumulate, and the error still grows with distance—exactly the three-dimensional version of the drift problem of Chapter 7. The LOAM of the next chapter gives a handsome answer: it registers not all the points but intelligently extracted edge features and planar features, and with a two-layer architecture of high-frequency odometry plus low-frequency mapping it strikes an excellent balance between real-time operation and accuracy. This is the technical backbone of 3D localization for autonomous inspection robots and self-driving cars.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 18

LOAM and LiDAR Odometry

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

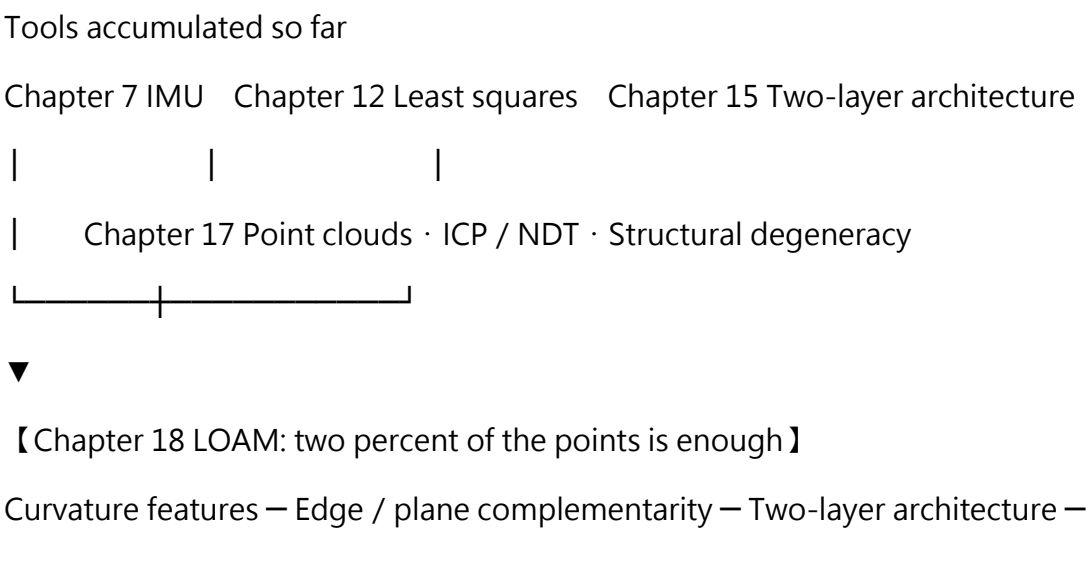
University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 17 assembled the tools for processing point clouds, but it also left an awkwardness: both ICP and NDT spend time finding correspondences, and a single 3D scan routinely holds tens of thousands of points, so registering frame by frame and accumulating is both slow and persistently drifting. The answer LOAM gives in this chapter sounds almost like a trick—do not use all the points, use only the two or three percent that have character. But there is a rigorous geometric reason behind the trick, and understanding that reason is the key to the whole of Part V.

The structure of this chapter echoes LOAM's two great contributions. Section 18.2 is about which points to pick, using curvature as the yardstick; Section 18.3 is about how to arrange the computation, using a two-layer high-frequency and low-frequency architecture—which students will find is the same design philosophy as Cartographer's local SLAM plus global SLAM in Chapter 15, incarnated in another dimension. From Section 18.4 onward come the descendants and applications of this architecture, leading all the way to the inspection projects of Part VII.

Figure 18-0 The Link Map of Chapter 18



compensation

▼

LeGO-LOAM ─ LIO-SAM ─ Fast-LIO (tightly coupled LiDAR-inertial)

▼

Part VI Visual SLAM Part VII Inspection and field applications

Learning Objectives

In the previous chapter we learned how to downsample and register point clouds, but accumulating frame-by-frame ICP over neighboring scans is both slow and persistently accumulating error. LOAM gives a handsome answer—it registers not all the points but intelligently extracted edge and planar features, and with a two-layer architecture of high-frequency odometry plus low-frequency mapping it balances real-time operation against accuracy. This is the technical backbone of 3D localization for autonomous inspection robots and self-driving cars. After studying this chapter, you should be able to:

1. Explain the central problem LOAM addresses and its place in the history of 3D LiDAR SLAM.
2. Explain how curvature (roughness) distinguishes edge features from planar features, and why registration with feature points alone suffices.
3. Explain the two-layer architecture of high-frequency odometry and low-frequency mapping, and why it can serve real-time operation and accuracy at once.
4. Compare the principal improvements of later systems such as LeGO-LOAM, LIO-SAM, and Fast-LIO.

5. Explain the motivation, the advantages, and the implementation essentials of tightly coupling LiDAR with an IMU.
6. Explain how 3D SLAM is applied to autonomous inspection robots.
7. Plan an inspection project set on a university campus (corridors, elevators, laboratories).

Core Concepts

中文名詞	English Term	Explanation
光達里程計與建圖	LOAM	The classic method of real-time LiDAR localization and mapping using edge and planar features.
光達里程計	LiDAR Odometry	The technique of inferring a robot's motion from consecutive LiDAR scans alone.
曲率 / 粗糙度	Curvature / Smoothness	The measure of how uneven the local neighborhood of a point is, used to classify features.
邊緣特徵	Edge Feature	A point of high curvature, lying at something sharp such as a corner, a column, or a tree trunk.
平面特徵	Planar Feature	A point of low curvature, lying on something flat such as a wall or the ground.
點到線距離	Point-to-Edge Distance	The error quantity used when registering edge features.
點到面距離	Point-to-Plane Distance	The error quantity used when registering planar features.
運動變形	Motion Distortion	The warping of one frame of point cloud caused by vehicle motion during the

中文名詞	English Term	Explanation
		LiDAR's revolution.
地面分離	Ground Segmentation	Treating ground points separately from non-ground points, the key to LeGO-LOAM.
兩步最佳化	Two-step Optimization	Solving some degrees of freedom from the ground plane first and the rest from edges afterward.
光達慣性里程計	LIO	An odometry method tightly coupling LiDAR with an IMU.
迭代擴展卡爾曼濾波	iEKF	The efficient tightly coupled estimation architecture adopted by Fast-LIO.

18.1 LOAM: LiDAR Odometry and Mapping

LOAM was proposed in 2014 by Ji Zhang and Sanjiv Singh at the CMU Robotics Institute, in a paper titled "LOAM: Lidar Odometry and Mapping in Real-time." It is the most influential single work in the field of 3D LiDAR SLAM—almost every mainstream system since has been built on its ideas.

The Problem It Addresses

The end of Chapter 17 left a difficulty: registering consecutive 3D scans frame by frame with ICP and accumulating meets two bottlenecks. The first is speed—with hundreds of thousands of points per frame, the nearest-neighbor search of ICP is a heavy burden and hard to keep real-time. When a scan contains a great many points, ICP may bear too high a computational cost. The second is accuracy—accumulate frame by frame and the error drifts continually.

LOAM's central insight is that not all the points are needed. What really supplies geometric constraint in an environment are the points with character—wall corners, columns, tree trunks (edges), and wall surfaces and floors (planes). Capture these feature points for registration and one can reach comparable or even better accuracy with less than one percent of the original point count.

LOAM's Two Great Contributions

- Feature extraction: classify points into edge features and planar features by curvature (roughness), greatly reducing the number of points that enter the computation (Section 18.2).

- Two-layer architecture: split the problem into a high-frequency but coarse odometry and a low-frequency but accurate mapping, two parallel tasks that serve real-time operation and accuracy together (Section 18.3).

LOAM splits the SLAM problem into two parallel tasks—odometry and mapping—and by extracting edge and planar features from the LiDAR scan and handling feature correspondence in a long-term optimization, achieves high accuracy at low computational cost in structured environments. This architecture of feature extraction plus KD-tree correspondence search has since been adopted as the core procedure by a great deal of LiDAR odometry research.

Its Place in History

Since Zhang and Singh proposed LOAM, LiDAR-based SLAM research has advanced rapidly; LOAM extracts edge and planar features by curvature and estimates a 6-degree-of-freedom pose by scan-to-map matching with a KD-tree, achieving computational efficiency and accuracy together by separating odometry and mapping into different execution frequencies.

For readers of this book, understanding LOAM is tantamount to holding the key to all of Part V: the LeGO-LOAM, LIO-SAM, and Fast-LIO of Section 18.4 are all its descendants, and research on university autonomous inspection robots builds on this same technical line.

18.2 Edge Features and Planar Features

This section covers LOAM's most recognizable design: how to pick the few genuinely useful feature points out of hundreds of thousands.

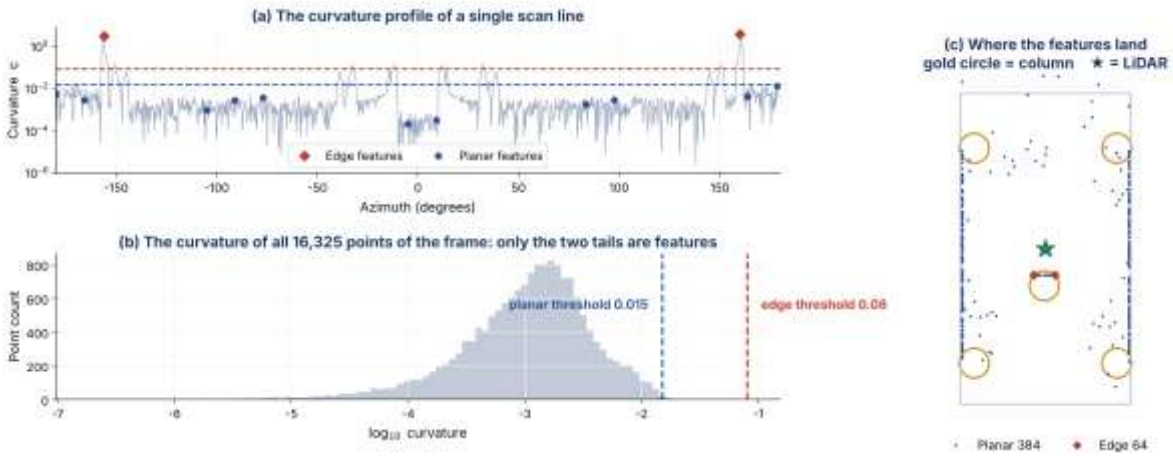
Curvature: the Yardstick for Identifying Features

LOAM classifies each point by a simple and ingenious measure—local roughness (commonly called curvature). The method is to take a point together with its neighbors on the same scan line and compute how far this point departs from the mean position of its neighbors.

- High roughness: the point is not collinear with the arrangement of its neighbors and lies at something sharp—this is an edge feature. Typical examples: wall corners, the edges of columns, tree trunks, and poles.
- Low roughness: the point lies nearly in a straight line with its neighbors, on something flat—this is a planar feature. Typical examples: wall surfaces, floors, ceilings, and large flat panels.

In each frame of the point cloud, only the few points of highest roughness need be taken as edge features and the few of lowest roughness as planar features. The illustrations in the LeGO-LOAM paper show this process clearly: after segmentation of the raw point cloud, edge and planar features are marked in blue and yellow respectively.

Figure 18-1 Curvature as a Yardstick: from One Scan Line to a Handful of Feature Points



This figure comes from a genuine simulation: a corridor 7 meters wide and 80 meters

long, with a colonnade, door frames, and a square column along it, scanned for one frame by a 32-beam LiDAR for 32,603 valid points. Panel (a) takes one of the scan lines and plots the curvature computed by Equation 1 against azimuth (the vertical axis on a logarithmic scale). For most of its length the curve murmurs along near 10^{-3}—that is the flat wall surface; and each time a column or a door frame is scanned a spike appears, leaping two orders of magnitude in an instant.

Panel (b) plots the curvature of all the points of the whole frame as a histogram, showing a single-peaked distribution with its peak near 10^{-3}. What LOAM wants is not this peak but the two tails of the distribution: the right tail (high curvature) is the edge features and the left tail (low curvature) the planar features. The great lump in the middle, which holds the overwhelming majority of the points, contributes almost nothing further to registration—this is the most direct visual evidence for what Case 18-1 calls the redundancy of most points.

Panel (c) verifies whether this screening really picks the right things. The red diamonds are edge features, and they can be seen falling accurately on the columns circled in gold and at the positions of the door frames and the square column; the small blue dots are planar features, spread evenly over the two long walls and the floor. The final numbers: 64 edge features and 384 planar features, together only 2.7 percent of the original point count. In other words, LOAM throws away 97 percent of the data, and what it throws away is precisely what could supply no new information.

This book also measured the price of that reduction. On the same scan, point-to-plane registration with all 32,513 points gave an angular error of 0.031 degrees, a translation error of 1.1 millimeters, and a time of 0.18 seconds; with only 768 planar features (2.4 percent), the angular error was 0.042 degrees, the translation error 1.3 millimeters, and

the time 0.038 seconds. Accuracy is barely lost while the time falls to a fifth. This is the fundamental reason LOAM can run in real time.

The Curvature Thresholds and Those Two Rejections

The thresholds in this figure are $c > 0.08$ for edges and $c < 0.015$ for planes, with at most 2 edge points and 4 planar points taken from each subregion of each scan line, so that features do not crowd into one place (the even distribution of Section 18.2).

The original LOAM paper adds two rejections, which this figure also implements: first, points on an occlusion boundary (where the range jumps between neighbors, the farther side being in fact occluded and bound to look different in the next frame); second, points whose scan line is nearly parallel to the surface (where the measurement is extremely unstable).

The rejection rate is about 8 percent. Without these two, false spikes caused by measurement noise on a distant wall are mistaken for edge features—exactly what happened in this book's first implementation, with edge points scattered over a bare wall rather than on the columns.

The thresholds must be adjusted to the LiDAR's beam count and to the site. The fewer the beams and the coarser the angular resolution, the noisier the curvature and the wider the thresholds must be.

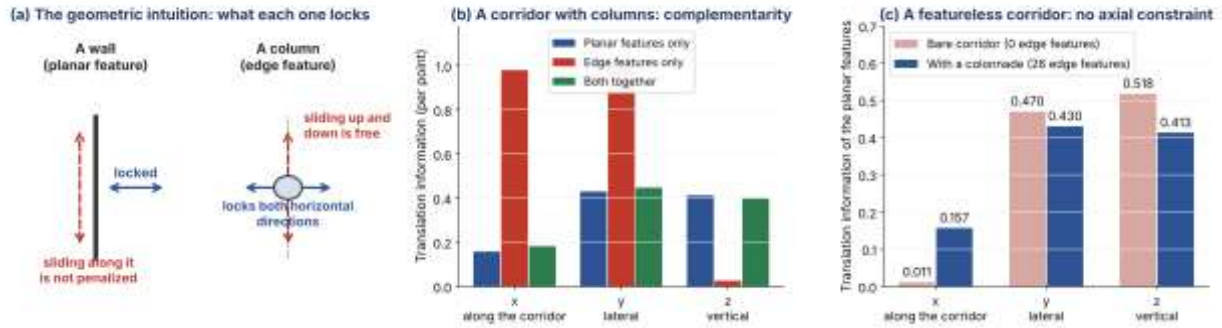
Why Both Kinds of Feature Are Needed

This is the most important idea in the section. The geometric constraints the two kinds of feature provide are complementary:

- Planar features constrain motion perpendicular to the plane. A wall can tell you how far you are from the wall but not how far you have walked along it—slide along the wall and the planar constraint does not change at all.
- Edge features constrain motion perpendicular to the edge line. A column pins your position in the two horizontal directions but has no say over motion up and down along the column.

Only by combining the two can all six degrees of freedom be pinned down. This also explains the structural degeneracy mentioned in Section 17.3: in a long corridor the walls all supply planar constraints in the same direction and edge features along the corridor are lacking, so the displacement along the corridor cannot be determined.

Figure 18-2 Why Neither Kind of Feature Can Be Spared: the Directions They Lock Are Exactly Complementary



This is the most important idea in the chapter, and it can be quantified exactly. The method is to write the sensitivity of the registration error to pose as an information matrix—the greater the information in some direction, the more a small departure in that direction raises the error, and so the more tightly that direction is locked. Panel (a) first gives the geometric intuition: a wall can lock only the one direction perpendicular to it, and sliding along the wall surface is not penalized at all; a column locks the two horizontal directions but has no say over sliding up and down along it.

Panel (b) is the measurement. With planar features alone, the information along the corridor is 0.157 while laterally it is 0.430 and vertically 0.413—along the corridor is clearly the weakest link. With edge features alone the situation is precisely reversed: the two horizontal directions are 0.980 and 0.994, all but saturated, while the vertical is only 0.026, which amounts to no constraint at all. The shapes of the two sets of numbers are complements of each other, which is the quantified version of Section 18.2's statement that the geometric constraints the two kinds of feature provide are complementary.

Panel (c) then pushes the conclusion to an extreme and answers the question left over from Chapter 17 along the way. Remove all the columns and door frames from the corridor, leaving a purely empty passage, and recompute: the information of the planar features along the corridor is only 0.011, against 0.470 laterally and 0.518 vertically—a difference of more than forty-fold. More telling still, in this empty corridor the number of edge features found by curvature screening is zero: the environment holds no sharp structure whatever that could supply an axial constraint.

This is the exact description, in LOAM's framework, of the structural degeneracy of Section 17.3: it is not that the algorithm is not good enough but that the geometric information itself does not exist. And put the colonnade back and 26 edge features appear at once, while the information of the planar features along the corridor rises from 0.011 to 0.157 (the door frames and the square column contribute surfaces perpendicular to the corridor). This also foreshadows the conclusion of Section 18.5—when the environment itself can supply no constraint, the only recourse is to take it from another sensor, and the IMU is that source.

How to Read the Information Number

It is the average constraint strength supplied by each feature point, dimensionless and between 0 and 1. A value of 1 means the direction is completely locked and 0 that it is completely free.

The information matrix of planar features is $\Sigma\, n\, n^T$ (n being the surface normal): whichever way the normal points is the direction locked.

The information matrix of edge features is $\Sigma\, (I - u\, u^T)$ (u being the direction of the edge line): all directions perpendicular to the line are locked. The columns in this figure are all vertical, so u is almost always the z axis and what is locked is the horizontal plane.

This analysis is also the basis of degeneracy detection in modern systems: compute the smallest eigenvalue of the information matrix in real time, and once it falls below a threshold the system knows it is walking into a degenerate scene and can raise its trust in the IMU (see Section 18.7).

Even Distribution: Avoiding Clustered Features

LOAM has one more practical detail: divide each scan line into several subregions and select a fixed number of feature points within each. This ensures that features are distributed evenly across the whole field of view rather than all clustered in one corner—if features cluster on one side the geometric condition of the registration is poor (much the same idea as the DOP of GNSS in Chapter 5).

The Definition of the Registration Error

Once features are extracted, how they are registered also depends on the feature type:

- Edge features: find the corresponding edge line in the previous frame (or in the map), determined by two neighboring edge points, and define the error as the perpendicular distance from the point to that line.
- Planar features: find the corresponding plane in the previous frame (or in the map), determined by three neighboring planar points, and define the error as the perpendicular distance from the point to that plane.

Minimize the sum of squared errors over all feature points and the best pose transformation follows—which is again the nonlinear least squares problem of Chapter 12. Students should feel by now that the mathematical tools of Part III appear again and again in every concrete system.

18.3 High-Frequency Odometry and Low-Frequency Map Optimization

LOAM's second core innovation is to split SLAM into two tasks that run in parallel at different frequencies. This architectural design is a classic and deserves careful study.

The Division of Labor between the Two Tasks

- LiDAR odometry (high frequency): compare the features of two neighboring scans and quickly estimate how far I moved in this tenth of a second. It uses only two frames, so it is light and fast but lower in accuracy and subject to drift.
- LiDAR mapping (low frequency): match the current scan against the accumulated global map for a finer registration. The map holds a great deal of historical information so the registration is more accurate, but it is heavy and runs at a low frequency.

In LOAM's original design the mapping module projects the point cloud onto the map at a frequency of 1 Hz; finally the pose transformations published by the two

algorithms are integrated to produce an output at about 10 Hz, representing the pose of the LiDAR relative to the map.

Figure 18-3 LOAM's Two-Layer Architecture

【LiDAR odometry】high frequency (about 10 Hz)

Match features between two neighboring frames → quickly estimate the relative motion

Characteristics: real-time, lightweight, lower in accuracy, subject to drift

↓ supplies the motion estimate and the initial guess

【LiDAR mapping】low frequency (about 1 Hz)

Match the current scan's features against the global map → fine registration

Characteristics: accurate, heavy to compute, low in frequency

↓

【Pose integration】

Integrate the two transformations → output an accurate pose at about 10 Hz

Why This Design Is Clever

The beauty of this architecture is that it keeps real-time operation and accuracy from conflicting:

1. High-frequency odometry meets the need of real-time control. A robot's motion control loop needs a high-frequency pose and cannot wait a full second for an update.

2. Low-frequency mapping takes charge of removing drift. It has ample time for a fine registration and pulls back the error accumulated by the high-frequency odometry.
3. The high-frequency odometry supplies the low-frequency mapping with a good initial guess. This is exactly what Section 17.3 stressed—ICP-type methods need a good initial value to converge, and odometry provides it.

Students should notice that this architecture of a fast but coarse front end plus a slow but accurate back end is cut from the same cloth as Cartographer's local SLAM and global SLAM subsystems in Chapter 15. This is no coincidence but a general pattern in robot system design: treat problems of different natures on different timescales.

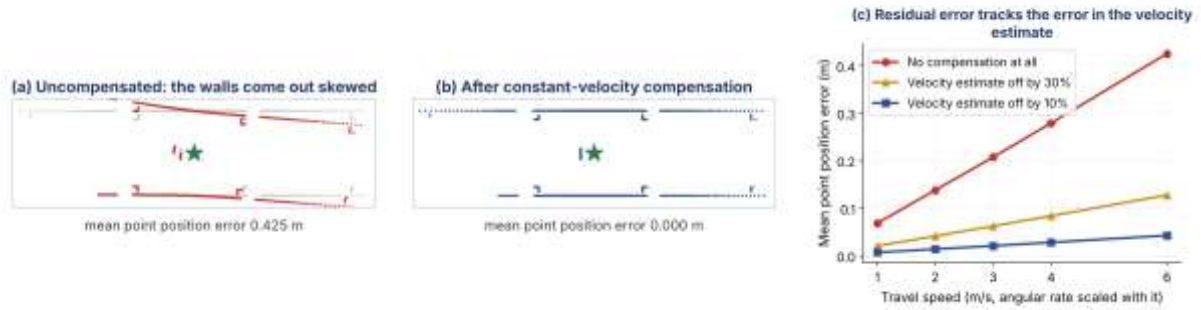
Motion Distortion Compensation

One further problem peculiar to 3D LiDAR must be handled. A mechanical LiDAR takes about 0.1 seconds (10 Hz) to complete one revolution, and during that time the vehicle itself is moving. The result is that the points of one frame of point cloud were in fact measured at different positions—the cloud is thereby warped, which is called motion distortion (mentioned in Section 8.4).

LOAM handles it by using the motion estimated by the odometry to project each point, according to the instant at which it was measured, back into the coordinate frame of the start of the sweep, removing the distortion. This is exactly why Section 17.1 mentioned that a point cloud needs timestamps. Later systems such as LIO-SAM also require explicitly that when the LiDAR spins at 10 Hz, each point's timestamp should vary between 0 and 0.1 seconds.

Figure 18-4 Motion Distortion: the Vehicle Moves While the LiDAR Completes a

Revolution



A mechanical LiDAR takes about 0.1 seconds to complete a revolution, and within that 0.1 seconds the vehicle itself is moving and turning. The result is that the points in one frame of point cloud were in fact measured at different poses. Take them naively as measurements made at one instant from one pose and the whole cloud is warped. Panel (a) simulates the phenomenon: the vehicle advances at 6 meters per second while turning at 1.6 radians per second; light gray is the true geometry and red the uncompensated point cloud—the two long walls are clearly skewed, with each point departing from its true position by 0.570 meters on average.

Panel (b) is after compensation. The method is exactly as described in Section 18.3: use a constant-velocity model to work out the pose at each point's measurement instant and project it back into the frame of the start of the sweep. The blue cloud coincides exactly with the gray truth and the mean error returns to zero. An honest note is in order here: it returns to zero because the velocity used for compensation is exactly the velocity used in the simulation—an ideal case, meant to verify that the compensation formula itself is written correctly.

The truly interesting panel is (c). The red line is the mean point position error with no compensation at all, and it is proportional to speed: 0.062 meters at 1 meter per second and 0.405 meters at 6. The amber and blue lines are the residual errors when the

velocity estimate is off by 30 percent and 10 percent respectively. The three lines display a clean proportionality: the residual error after compensation is about equal to the percentage error of the velocity estimate times the error with no compensation at all. Get the velocity 10 percent wrong and the residual error is 10 percent of the uncompensated value.

This proportionality ties Sections 18.3 and 18.5 together. It shows that the quality of motion compensation depends entirely on the quality of the velocity estimate, and the velocity estimate is exactly what an IMU does best—an update rate of several hundred hertz and extreme accuracy over short intervals. This is why virtually every modern LiDAR system carries an IMU: not only to supply the initial value for registration but to ensure that every frame of point cloud is already clean and undistorted before it enters registration at all.

Three Practical Points About Motion Distortion

The faster the speed and the sharper the turn, the worse the distortion; and turning matters more than driving straight, because rotation displaces distant points in proportion to their range. At 6 m/s the largest single-point error in this figure reaches 2.36 meters.

Compensation requires a timestamp for every point. This is exactly the use of the timestamp field mentioned in Section 17.1, and one of the reasons Case 18-3 says a missing required field stops the system from running.

A solid-state LiDAR has no rotating mechanism, but its scan still takes time and it suffers distortion just the same, only in a different pattern.

The constant-velocity model is the simplest assumption. It loses accuracy

under sharp acceleration or a sharp turn, and then the angular rate and acceleration measured by an IMU are more reliable than anything the model infers.

18.4 An Introduction to LeGO-LOAM, LIO-SAM, and Fast-LIO

LOAM opened the road, and the decade and more since has brought a great many improved systems. This section introduces three of the most representative descendants, each of which addresses a different weakness of LOAM.

1. LeGO-LOAM: Lightweight and Ground-Optimized

LeGO-LOAM stands for Lightweight and Ground-Optimized LiDAR Odometry and Mapping, designed for real-time 6-degree-of-freedom pose estimation on ground vehicles. It is lightweight, achieving real-time pose estimation on a low-power embedded system; and it is ground-optimized, because it makes good use of the presence of the ground plane in its segmentation and optimization steps.

Its two key improvements are:

- Point cloud segmentation and ground separation: segment the cloud first to filter out noise, then extract features to obtain discriminative planar and edge features. Segmentation effectively removes unstable points such as vegetation and noise—which matters especially outdoors.
- Two-step optimization: use a two-step Levenberg–Marquardt optimization, solving different components of the 6-degree-of-freedom transformation with planar and with edge features separately. Specifically, first use the planar features extracted from ground points to solve for the vertical displacement

and the roll and pitch angles, then use the edge features of the segmented cloud to estimate the horizontal displacement and the yaw angle, with the result of the first step as a constraint.

Measurements show that LeGO-LOAM achieves accuracy comparable to or better than LOAM at lower computational cost on ground vehicle datasets over varied terrain. It is optimized in particular for a 16-beam LiDAR mounted horizontally on a ground vehicle and assumes that a ground plane is always present in the scan—an assumption that limits its range of application but is also exactly what makes it efficient.

2. LIO-SAM: Tightly Coupled LiDAR-Inertial Odometry with a Factor Graph

LIO-SAM's innovation lies in tightly coupling the IMU with a factor graph and a Bayes tree, achieving motion-aware submap matching and integrating drift-aware loop closure detection.

- Tightly coupled IMU: add IMU preintegration (Chapter 7) to the graph as a factor, greatly improving robustness under fast motion.
- Factor graph architecture: use the GTSAM and iSAM2 incremental solvers introduced in Chapter 12 to fuse constraints from LiDAR, IMU, GPS, and loop closure efficiently.
- Loop closure detection: its loop closure function is adapted from LeGO-LOAM and is ICP-based; because ICP runs rather slowly, the authors recommend playing back at the original speed when testing.

An implementation limitation worth noting: like the original LOAM implementation, LIO-SAM can only be used with a nine-axis IMU (one able to supply roll, pitch, and yaw estimates), and it currently supports mechanical LiDARs only. Details of this kind are critical in project work, and students should always confirm them before

choosing hardware.

3. The Fast-LIO Family: Efficiency Taken to Its Limit

Fast-LIO takes another road: it performs no feature extraction, uses the raw point cloud directly, and estimates by tight coupling with an iterated extended Kalman filter (iEKF). Its key innovation is an efficient formula for computing the Kalman gain that makes the computational complexity depend on the dimension of the state rather than on the number of measurements, greatly raising its speed.

Fast-LIO2 goes further and introduces an incremental KD-tree (ikd-Tree) data structure that can update the map incrementally and efficiently, reaching very high processing rates and suiting platforms with limited computing resources and vehicles in fast motion (drones, say). This line proves that, given a good motion prior from an IMU, one can be both fast and accurate without feature extraction at all.

System	Year	Key Improvement	Suitable Scenes
LOAM	2014	Feature extraction + two-layer architecture	General purpose; the founding work
LeGO-LOAM	2018	Ground separation, two-step optimization, lightweight	Ground vehicles, embedded platforms
LIO-SAM	2020	Tight IMU coupling, factor graph, multi-source fusion	Sites needing high robustness and loop closure
Fast-LIO2	2021	Feature-free, iEKF, incremental KD-tree	Computation-limited platforms, fast motion

Table 18-1 The Main Systems of the LOAM Family Compared

18.5 Tightly Coupled LiDAR and IMU

Two of the three systems in Section 18.4 emphasize the IMU. This section explains why LiDAR SLAM depends more and more on the IMU, and what tight coupling means.

Three Weaknesses of LiDAR Used Alone

- Failure under fast motion: pure LiDAR methods degrade under fast dynamic motion; consecutive scans differ too much and registration struggles to converge (the old problem of Section 13.3, that ICP needs a good initial value).
- Geometrically degenerate environments: in geometrically degenerate environments such as long corridors or open spaces, insufficient geometric constraint makes pose estimation fail. This is the structural degeneracy of Section 17.3 appearing once again.
- Motion distortion: as described in Section 18.3, vehicle motion during a revolution warps the point cloud, and motion information is needed to compensate it.

The IMU Fills Exactly These Three Gaps

Recall Chapter 7: an IMU has a very high update rate (several hundred hertz), is extremely accurate over short intervals, and is entirely indifferent to the structure of the environment. It can therefore:

4. Supply a high-frequency motion estimate between two LiDAR frames as an excellent initial value for registration, solving the fast motion problem.
5. Supply an independent constraint in the geometrically degenerate direction—in a corridor the LiDAR cannot say how far it has traveled, but integrating the IMU's acceleration yields an estimate.
6. Supply an accurate attitude at the measurement instant of every point, for use in motion distortion compensation.

Conversely, the LiDAR gives the IMU what it most needs: a non-drifting pose observation with which to estimate and compensate the IMU's bias continually (Section 7.3). LiDAR-inertial odometry (LIO) has therefore drawn a great deal of attention; it fuses LiDAR measurements and IMU data tightly, combining LiDAR's robustness to changes in light with its ability to capture dense geometric information.

Loose Coupling vs. Tight Coupling

Section 7.6 discussed this pair of concepts in the context of visual-inertial SLAM, and it applies in full to the LiDAR field:

- Loose coupling: LiDAR SLAM and the IMU each compute a pose and the two are then fused with weights. Simple to implement, but the information has already been compressed and accuracy is limited.
- Tight coupling: put the IMU preintegration constraints and the LiDAR feature registration errors into one and the same optimization problem (or one and the same filter) and solve them together. Markedly better in accuracy, and the unanimous choice of modern systems.

LIO-SAM realizes tight coupling with a factor graph (the optimization road, Chapter 12) and Fast-LIO with an iEKF (the filtering road, Chapter 11)—different routes to the same destination, confirming once again that Chapters 11 and 12 are precisely the two trunk lines of SLAM.

Implementation Essentials

- Time synchronization: the timestamps of LiDAR and IMU must be aligned precisely, and an error of a few milliseconds affects accuracy appreciably (the engineering detail stressed repeatedly in Chapters 5 and 10).

- Extrinsic calibration: the relative mounting position and orientation of the IMU and the LiDAR must be measured accurately (the extrinsics of Chapter 10), or the fused result carries a systematic bias.
- IMU specification: as noted above, some systems require a nine-axis IMU. The quality of the IMU (the grade table of Chapter 7) also bears directly on how long it can hold out in a degenerate scene.

18.6 3D SLAM for Autonomous Inspection Robots

One of the most important practical applications of 3D LiDAR SLAM is the autonomous inspection robot. This section explains the requirements and design considerations of such systems.

Why Inspection Needs 3D Rather Than 2D

- Uneven terrain: an inspection robot (a robot dog above all) must go up and down stairs, cross thresholds, and traverse ramps. A 2D map cannot describe changes in height, nor judge whether a given step can be cleared.
- Three-dimensional obstacles: the suspended piping of a factory, the top edge of a rack in a machine room, and tree branches all lie off the 2D scan plane. The height blind spot of Chapter 8 causes collisions directly at an inspection site.
- Three-dimensional positions of inspection targets: to record that the second pipe on the east wall of the third floor is corroded requires three-dimensional coordinates rather than a planar position.
- Asset modeling: the by-product of an inspection is often a three-dimensional model of the facility (the digital twin of Section 17.7), which has value in itself.

A Typical System Architecture

Figure 18-5 The 3D SLAM System Architecture of an Autonomous Inspection Robot

【Sensing layer】3D LiDAR + IMU + (optionally) camera, GNSS, wheel encoders

↓

【Front end】feature extraction (edge / plane) or direct point cloud registration

↓

【Tightly coupled estimation】LIO: LiDAR constraints + IMU preintegration → high-frequency 6-DoF pose

↓

【Back end】factor graph optimization + loop closure detection (Chapter 12)

↓

【Map】3D point cloud map → converted to an octree for navigation (Section 17.5)

↓

【Application layer】path planning, obstacle avoidance, navigation to inspection points, image / thermal data collection

Platform Choice: Wheeled, Tracked, or Quadruped

- Wheeled: highest in efficiency and best in endurance, but confined to level ground and unable to climb stairs. Suited to factory floors and corridor inspection.

- Tracked: able to cross larger obstacles and ramps, and common at disaster sites and in outdoor inspection.
- Quadruped (robot dog): strongest in adaptation to terrain, able to climb stairs and step over obstacles, and a popular choice for inspection in recent years. The price is shorter endurance and higher cost, and the violent vibration of its gait is a severe test for SLAM—which is exactly why tight IMU coupling is required.

The Special Requirements of Inspection Tasks

- Repetition: inspection is a periodic task, walking the same route every day. The pattern of mapping once and localizing thereafter is therefore particularly apt (Case 14-3 of Chapter 14), and data from different times can be compared to find changes (a newly appeared crack, say).
- Accurate positioning at inspection points: the robot must stop at a specific position and photograph an instrument or piece of equipment from a specific angle. The positioning error must be held within a few centimeters, and this is the source of 3D SLAM's accuracy requirement.
- Long running times: an inspection task may last several hours, placing very high demands on drift control and system stability—loop closure detection is therefore indispensable.
- Data association: the images, thermal images, and audio collected during inspection all need to be tagged with an accurate three-dimensional position so that problems can be located in the digital twin.

18.7 After LOAM: Four Directions in LiDAR-Inertial Odometry, 2024–2026

LOAM was published in 2014, and for more than a decade its two great designs—feature extraction and the two-layer architecture—have been the standard skeleton of LiDAR SLAM. But over the last two years both of these pillars have been seriously challenged, and the challenges point in the same direction: computation has become cheap, so the compromises made for real-time operation back then may no longer pay. This section gathers four directions.

1. Feature Extraction Is Being Abandoned

The first direction was already foreshadowed in Section 18.4's introduction to Fast-LIO: perform no feature extraction and use the raw point cloud directly. This line has become all but mainstream in recent years. The reason is pragmatic—extracting features in 2014 was a way of reducing hundreds of thousands of points to a few thousand, without which the computation could not keep up; but on today's processors and GPUs, voxelized registration on tens of thousands of points directly is entirely feasible, and dropping the feature extraction step drops a set of thresholds that need tuning along with failure modes such as a column too slender to be caught.

This is the same current as the KISS-ICP of Section 17.8: rather than design ingenious screening rules, do the most basic registration solidly. Yet the analysis of Figure 18-2 in this chapter has not gone out of date—feature-free methods face geometric degeneracy just the same, only they no longer describe it in the vocabulary of edges and planes but look directly at the covariance of the voxel distribution. The concept is the same, differently worded.

2. Global Consistency Returns to the Front: GLIM

The second direction is a correction to the two-layer architecture itself. LOAM's low-

frequency mapping does correct drift, but once the front end has fixed the pose of a frame it never looks back. GLIM, published in 2024 in Robotics and Autonomous Systems by Kenji Koide's team at the National Institute of Advanced Industrial Science and Technology, argues that this fix-it-and-never-change design is precisely the root of collapse in degenerate scenes.

GLIM's approach has two keys: first, the odometry end adopts fixed-lag smoothing (a five-second window) so that the poses of the last few seconds continue to be corrected rather than settled at once; second, global optimization minimizes the registration error between submaps directly rather than first compressing them into Gaussian constraints on a pose graph. Add GPU-accelerated voxelized GICP and the whole system takes 15.1 milliseconds per frame of preprocessing and 29.3 milliseconds of odometry on ordinary equipment.

The effect is clearest in degenerate scenes. The paper tests a solid-state LiDAR on eight degenerate sequences, where GLIM's absolute trajectory error falls between 0.017 and 0.389 meters while FAST-LIO2's range is 0.297 to 15.551 meters—a fortyfold difference on the worst sequence. This contrast is worth remembering: under good conditions the various systems differ little, and what really separates them is always the worst case.

3. Degeneracy Is No Longer Merely Endured but Actively Detected

The third direction follows directly from the analysis of Figure 18-2 in this chapter. Since degeneracy can be quantified by the smallest eigenvalue of the information matrix, it can be computed in real time: once the constraint in some direction is found insufficient, the system automatically lowers its trust in the LiDAR and raises the weight on the IMU, or simply declines to update in that direction. Degeneracy-aware LiDAR-inertial odometry of this kind has been quite an active topic around 2025.

For a student project this offers a direction of modest difficulty and real value: there is no need to write a complete LIO of your own, only to add a monitor of the information matrix to an existing system, plot the smallest eigenvalue against time, and compare it with the site the robot actually walked, which yields a map of where in this building SLAM will fail. That is directly useful for the inspection project of Chapter 19.

4. From LiDAR-Inertial to LiDAR-Inertial-Visual

The fourth direction brings the camera into the same tightly coupled framework. Systems of the FAST-LIVO2 kind add a direct photometric image error on top of Fast-LIO's iterated Kalman filter architecture, forming a tight coupling of LiDAR, inertial, and visual sensing. The motivation agrees exactly with the analysis of sensor complementarity in Part II: LiDAR gives geometry, the camera gives texture and semantics, and the IMU gives high-frequency motion, and their failure modes barely overlap.

Its significance for degenerate scenes is especially worth noting. A long corridor leaves the LiDAR unable to say how far it has traveled, but the posters on the walls, the door plates, and the floor texture are clear features to a camera—which fills exactly the 0.011 gap of Figure 18-2(c). In other words, the visual SLAM of Part VI is not merely another parallel route but also the rescue plan for this chapter's systems in degenerate scenes.

The Common Thread of the Four Directions

Computation (feature-free): the screening done for real-time operation back then may not pay on today's hardware.

Architecture (GLIM): the front end should not fix a pose at once, and keeping room for correction is what carries a system through a degenerate stretch.

Diagnosis (degeneracy awareness): degeneracy turns from a disaster discovered after the fact into a metric measurable in real time.

Sensing (LiDAR-inertial-visual): a gap one sensor cannot fill is filled by another whose failure modes do not overlap with it.

The common thread: the three fundamentals of this chapter—curvature, complementary constraints, and motion compensation—have not been retired but have become the standard by which a new system is judged. Any new method must answer the same three questions: which points are useful, which direction has no constraint, and how far the vehicle moved during the sweep.

Carrying Forward

This chapter has completed the core technology of 3D LiDAR SLAM. From the point cloud processing of Chapter 17 to the LOAM family here, students now possess the full knowledge needed to localize and map in a three-dimensional world: Figure 18-1 shows which points are worth keeping, Figure 18-2 shows why neither kind of feature can be spared, and Figure 18-4 shows that motion during a sweep must be compensated. These three things are also the starting points from which every later system—LeGO-LOAM, LIO-SAM, Fast-LIO, GLIM—made its own improvements.

But the figure 0.011 in Figure 18-2(c) is a problem this chapter cannot solve on its own: when the geometry of the environment itself supplies no constraint, no LiDAR algorithm, however good, can conjure information. Section 18.5 gave the first answer—the IMU—and Part VI will give the second—the camera.

Before that, the next chapter must first answer another question: what happens when these techniques are put into a robot that has to run every day for a year? Chapter 19

takes 3D SLAM into the field of autonomous inspection and handles three things that algorithm textbooks do not teach but that decide whether a system succeeds: the ordering of inspection points, repeat positioning accuracy, and long-term map maintenance—and this chapter's degeneracy analysis in Figure 18-2 becomes, there, the concrete consequence of one inspection point being photographed crooked every other visit.

Formulas and Procedures

1. Local Roughness (Curvature)

For the i-th point p_i, take the set S of neighboring points on the same scan line; its roughness is defined as:

$$c = \left(1 / \left(|S| \cdot \|p_i\|\right)\right) \cdot \left\| \sum_{j \in S} \left(p_i - p_j\right) \right\|$$

The meaning of the expression is that the difference vectors between p_i and all its neighbors are summed. If p_i lies on something flat, the difference vectors to the left and right neighbors cancel and c is small; if it lies at a sharp corner, the difference vectors add in the same direction and c is large. The $\|p_i\|$ in the denominator normalizes the influence of distance (points far away are spaced more widely and should not be mistaken for edges on that account).

- c greater than a threshold → edge feature
- c less than a threshold → planar feature

2. Point-to-Line and Point-to-Plane Distance

The error of an edge feature is the perpendicular distance from the point to the corresponding edge line. Let the current edge point be p and the corresponding line

be determined by two points a and b in the map:

$$d_edge = \| (p - a) \times (p - b) \| / \| a - b \|$$

The numerator is the area of the parallelogram formed by the three points (computed as a cross product), and dividing by the length of the base gives the height—which is the distance from the point to the line.

The error of a planar feature is the perpendicular distance from the point to the corresponding plane. Let the plane be determined by three points a, b, and c in the map, with normal $n = (b - a) \times (c - a)$:

$$d_plane = | (p - a) \cdot n | / \| n \|$$

3. LOAM's Optimization Objective

Combine the errors of all feature points and seek the pose transformation T that minimizes the total:

$$T^* = \text{argmin} \left(\Sigma\, d_edge(T \cdot p_i)^2 + \Sigma\, d_plane(T \cdot p_j)^2 \right)$$

This is a nonlinear least squares problem, solved iteratively by the Levenberg–Marquardt method (Chapter 12). Students can see that from the abstract theory of Chapter 12, through the 2D scan matching of Chapter 13 and the ICP of Chapter 17, to the LOAM of this chapter, it is one body of mathematics applied to different data.

4. The Complete LOAM Processing Flow

Figure 18-6 LOAM's Processing Flow for Each Frame

① Receive one frame of 3D point cloud (with a timestamp and ring number for each point)

↓

② Motion distortion compensation: project each point back to the start of the sweep using the motion estimate

↓

③ Compute the local roughness c of every point

↓

④ Select features by region: in each subregion take the points of highest roughness (edges) and lowest (planes)

↓

⑤【Odometry, 10 Hz】match features against the previous frame → quickly estimate the relative motion

↓

⑥【Mapping, 1 Hz】match features against the global map → a fine pose, and update the map

↓

⑦ Integrate the two outputs → an accurate pose at about 10 Hz + a global 3D map

Cases

Case 18-1 Why Using One Percent of the Points Is More Accurate

When beginners hear that LOAM registers with only a few feature points, the instinctive reaction is usually: discard so much data and will accuracy not fall? The answer is unexpected—not only does it not fall, it often rises.

There are three reasons. First, most points are redundant: a flat wall carries tens of thousands of points, but the geometric constraint they supply is in fact only one—how far you are from this wall. Express that constraint with ten thousand points or with a hundred and the result is all but the same, while the computation differs a hundredfold.

Second, some points are harmful: leaves of vegetation, a fluttering plastic bag, a person walking by—these points are no longer in place in the next frame, and forcing a registration on them only injects error. LeGO-LOAM's point cloud segmentation step exists precisely to filter out unstable points of this kind.

Third, feature points are carefully chosen: points of very high or very low curvature happen to be the points most easily recognized correctly in the next frame—exactly the reason visual SLAM in Chapter 9 prefers corners ("not one point can be recovered on a white wall").

This case offers students a lesson beyond the technical: more data does not mean a better result, and what matters is the quality and representativeness of the data. The principle applies throughout machine learning, statistics, and engineering judgment.

Case 18-2 The Value of the Ground: LeGO-LOAM's Ingenuity

LeGO-LOAM has "ground-optimized" in its name, and this seemingly plain design is in fact very clever.

Consider: what is the ground to a robot driving on it? It is a planar feature that is always present, extremely stable, and enormous in area. And the constraint it supplies

is very definite—the ground tells the robot exactly how high above it the body is and whether the body is tilting forward, back, or to one side, that is, the three degrees of freedom of vertical displacement, roll, and pitch.

LeGO-LOAM therefore splits the six degrees of freedom into two groups solved separately: first use the planar features of ground points to solve for the vertical displacement and the roll and pitch angles, then use the other edge features to estimate the horizontal displacement and the yaw angle, taking the result of the first step as a constraint. The benefit is that each step has fewer unknowns to solve, so the problem converges better, computes faster, and is more stable.

This case demonstrates an important kind of engineering thinking: make good use of the prior knowledge of the site. That a ground vehicle is certainly on the ground is a free and reliable assumption, and not to use it is to waste it. The price, of course, is generality—the assumption does not hold at all on a drone. This again echoes the observation of Chapter 15: the design orientation of a technology is decided by the needs of its application.

Case 18-3 The Devil in the Details of Implementation: the Pitfalls of Hardware Specifications

Many students eagerly download LIO-SAM or LeGO-LOAM for a project only to be stuck at not being able to get it running. The reason is usually not the algorithm but a mismatch in hardware specification. Here are several real pitfalls:

- Too few IMU axes: like the original LOAM implementation, LIO-SAM can only be used with a nine-axis IMU (one able to supply roll, pitch, and yaw estimates). A student who bought a six-axis IMU is stopped outright.

- Unsupported LiDAR type: only mechanical LiDARs are currently supported. Solid-state LiDARs (Chapter 8) have a different data format and require extra adaptation.
- Missing required fields: the system needs a timestamp and a ring number for every point. The official documentation states explicitly that when the LiDAR spins at 10 Hz each point's timestamp should lie between 0 and 0.1 seconds, while the ring number indicates which channel of the sensor the point belongs to and is used to organize the cloud correctly. If the driver does not output these fields they must be added by hand.
- Optimization for a particular LiDAR: LeGO-LOAM is optimized in particular for a 16-beam LiDAR mounted horizontally on a ground vehicle and assumes a ground plane is always present in the scan. Used in other configurations, its performance is discounted.

The pedagogical point of this case is that an open-source system is not download-and-run. Reading the hardware requirements section of the official documentation is required homework before a project begins. The habit saves weeks of frustration—which is also why every chapter of this book sets out prerequisites and common failures in detail.

Hands-On Activities

Exercise 18-1 LiDAR Odometry and Feature Extraction

Purpose: implement curvature feature extraction by hand, observe the distribution of edge and planar features, and run a complete LiDAR odometry system to understand the roles of drift and loop closure. This exercise works mainly from public datasets and needs no 3D LiDAR hardware.

Part One: Implementing Feature Extraction (everyone)

1. Download a segment of public 3D LiDAR data (KITTI, or the campus dataset provided officially with LeGO-LOAM) and take a single frame of point cloud from it.
2. Implement the roughness computation of Equation 1 in Python: for each point, take several neighbors on each side with the same ring number and compute the value of c.
3. Classify by c: take the highest 2 percent as edge features and the lowest 20 percent as planar features, display them in different colors (Open3D will serve), and observe where in the environment each class falls.
4. Verify the intuition: do the edge features cluster on columns, tree trunks, and wall corners? Do the planar features cluster on the ground and the walls? If not, adjust the neighbor count and the thresholds and try again.
5. Count what proportion of the original points the feature points make up, to get a feel for LOAM's reduction.

Part Two: Running LiDAR Odometry (everyone, or university level)

6. Install LeGO-LOAM or A-LOAM in a ROS environment (taking care to check the hardware and data format requirements of Case 18-3).
7. Play back the dataset and observe in RViz in real time: the feature point cloud, the estimated trajectory, and the 3D map accumulating step by step.
8. Observe the drift: let the data play through one large loop, compare the positions of start and end, and quantify the accumulated drift.

9. Turn on loop closure detection and run again, observing how the whole trajectory and map are straightened when the loop closure fires (the striking moment of Chapters 12 and 15).

Part Three: System Comparison and a Degeneracy Experiment (university level)

10. Run LeGO-LOAM, LIO-SAM (if IMU data is available), and Fast-LIO on the same segment of data, and compare their trajectory accuracy, computational burden, and map quality.
11. Use the wall thickness method of Case 17-1 in Chapter 17 to compare the quality of the three maps quantitatively.
12. Degenerate scene experiment: find a segment of data dominated by a long corridor (or collect one yourself) and compare the performance of pure LiDAR against LiDAR plus IMU, verifying the argument of Section 18.5.
13. If IMU data is available, deliberately disable the IMU fusion and run again, quantifying the improvement the IMU brings.

Discussion Questions

1. What percentage of the original points are your feature points? If you doubled that proportion, would the registration accuracy improve? By how much would the computation time rise?
2. Examine the features you extracted: in areas thick with vegetation or clutter, is the distribution of features reasonable? Does this bear out why LeGO-LOAM segments the point cloud first?
3. How many meters is the accumulated drift over the large loop? What percentage of the total distance traveled is that? How much did loop closure correction improve it?

4. (University level) In the long corridor scene, in which direction is the error largest? Is this the same problem as the 2D corridor predicament of Section 13.6 and the structural degeneracy of Section 17.3?
5. (University level) Using Table 18-1, which system would you choose for a university inspection robot dog? What factors enter the decision?

Extended Challenge (university level)

First: implement the simplest possible LiDAR odometry—use planar features only, register consecutive pairs of frames with point-to-plane ICP (Chapter 17) and accumulate the poses, compare the trajectory with the dataset's ground truth, and plot the curve of error growing with distance. Then add edge features and observe how much the accuracy improves, verifying for yourself the argument of Section 18.2 that the two kinds of feature are complementary. Second: collect a segment of real data in a university corridor (following the method of Stage One in Section 19.8), run this chapter's systems on it and assess the quality, as a preliminary study for Project Four of Chapter 31.

Further Reading

1. J. Zhang and S. Singh, "LOAM: Lidar Odometry and Mapping in Real-time," Robotics: Science and Systems, 2014. (The original LOAM paper, the main basis of this chapter, from the CMU Robotics Institute.)
2. T. Shan and B. Englot, "LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping on Variable Terrain," IROS, 2018. (The original paper on ground optimization and two-step optimization.)

3. T. Shan et al., "LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping," IROS, 2020. (The representative work on factor graph tight coupling.)
4. W. Xu and F. Zhang, "FAST-LIO: A Fast, Robust LiDAR-inertial Odometry Package by Tightly-Coupled Iterated Kalman Filter," IEEE RA-L, 2021. (And FAST-LIO2, IEEE T-RO, 2022.)
5. The LeGO-LOAM and LIO-SAM open-source projects (GitHub). (With official datasets, hardware requirement notes, and parameter settings; required reading for implementation.)
6. The A-LOAM open-source project (GitHub, HKUST Aerial Robotics). (A clean reimplementation of LOAM with readable code, suited to study in teaching.)
7. The official KITTI Vision Benchmark Suite website. (The standard benchmark dataset for self-driving 3D LiDAR, with ground truth trajectories.)

Summary

1. LOAM was proposed by Zhang and Singh in 2014, and its central insight is that not all the points are needed; it splits SLAM into the two parallel tasks of odometry and mapping and, by extracting edge and planar features, achieves high accuracy at low computational cost in structured environments, becoming the common foundation of later LiDAR SLAM research.
2. Features are classified by local roughness (curvature): high curvature gives edge features (columns, wall corners) and low curvature planar features (walls, floors); the two supply complementary geometric constraints—planes constrain motion perpendicular to the surface and edges motion perpendicular to the line—and without either, the degrees of freedom cannot be fully locked.

3. The two-layer architecture is LOAM's second innovation: high-frequency odometry (about 10 Hz) supplies a real-time pose and a good initial value, while low-frequency mapping (about 1 Hz) registers finely against the global map to remove drift, and the two are integrated into a high-frequency, accurate pose output. Motion distortion compensation must also be performed.
4. LeGO-LOAM filters out noise by point cloud segmentation and makes good use of the ground plane in a two-step Levenberg–Marquardt optimization, running in real time on a low-power embedded system and achieving comparable or better accuracy at lower computational cost, but assuming a ground plane is always present in the scan.
5. LIO-SAM achieves tight IMU coupling and drift-aware loop closure integration with a factor graph and a Bayes tree; the Fast-LIO family takes the road of no feature extraction and tight coupling with an iterated extended Kalman filter, suiting computation-limited platforms and fast motion.
6. Pure LiDAR fails under fast motion and in geometrically degenerate environments (long corridors, open ground), and the IMU fills exactly these three gaps (supplying an initial value, an independent constraint in the degenerate direction, and motion distortion compensation), while the LiDAR returns non-drifting observations that correct the IMU's bias; tight coupling beats loose.
7. Autonomous inspection needs 3D rather than 2D because it must handle uneven terrain, three-dimensional obstacles, three-dimensional inspection targets, and asset modeling; the platform may be wheeled, tracked, or

quadruped, and the gait vibration of a quadruped makes tight IMU coupling a necessity.

This chapter has completed the core technology of 3D LiDAR SLAM. From the point cloud processing of Chapter 17 to the LOAM family here, students now possess the full knowledge needed to localize and map in a three-dimensional world. In the next chapter we take these techniques into a real site—the autonomous inspection robot of Chapter 19—handling the ordering of inspection points, repeat positioning accuracy, and long-term map maintenance, the three things that decide whether a system succeeds, and so closing Part V.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 19

Autonomous Inspection Robots and Field Practice

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapters 17 and 18 assembled the technology of three-dimensional LiDAR SLAM: how to process a point cloud, how to pick features, how to estimate a pose, and how to compensate motion during a sweep. This chapter asks another question—what happens when these techniques must run by themselves for three hours a day in the same building, every day for a year.

The answer is that technical metrics are no longer the only thing that matters. Whether an inspection robot succeeds often turns not on how accurate its SLAM is but on three more mundane things—whether the inspection points are well ordered, whether it can stop at the same position every day, and whether a map built three months ago still works today. These three things are the subjects of Sections 19.4, 19.5, and 19.6, and the questions the chapter's three experimental figures set out to answer.

The role of this chapter in the book is symmetrical with that of the cleaning robot in Chapter 16: Chapter 16 was the practical landing of 2D SLAM, and this chapter is the practical landing of 3D SLAM. Both demonstrate the same thing—put an algorithm into a product that must be used every day and the real difficulties only begin.

Figure 19-0 The Link Map of Chapter 19

The technical foundation of Part V

Chapter 17 Point cloud processing Chapter 18 LOAM and feature registration

| |

└──────┬──────┘

▼

【Chapter 19 Autonomous inspection: making the technology run every day for a year】

Route planning ─ Repeat positioning ─ Map maintenance ─ From data to a digital twin

▼

Part VI Visual SLAM Part VII Drones and edge AI

Part VIII ROS implementation Part IX Semantic navigation

Learning Objectives

Inspection is the most mature commercial application of 3D SLAM and the site that best ties together the first eighteen chapters of this book. Starting from requirements, this chapter treats in turn the technology stack of an inspection robot, platform selection, route planning, repeat positioning, and long-term maintenance, closing with the state of research and industry in Taiwan and with an executable campus project. After reading this chapter, you should be able to:

1. Explain the problems the automation of inspection sets out to solve, and how it differs fundamentally from an ordinary navigation task.
2. Describe the technology stack of an inspection robot and point out which chapters of this book each layer corresponds to.
3. Compare the suitable sites and the costs of the four platforms: wheeled, tracked, quadruped, and aerial.

4. Explain the essential nature of the inspection route planning problem and estimate the effect of ordering on endurance.
5. Explain why repeat positioning accuracy is the key metric of inspection, and how it relates to the geometry of the site.
6. Explain why map maintenance is necessary in long-term operation, and design a map update strategy.
7. Give examples of Taiwan's research and industrial achievements in inspection robots and plan a campus inspection project on that basis.

Core Concepts

Term	Meaning	Related Chapters
Waypoint	A position the robot must reach and at which it must collect data in a particular attitude	19.4, 19.5
Repeat positioning accuracy	How widely the pose scatters over repeated returns to the same inspection point	19.5, Chapter 11
The difference between coverage and inspection	Coverage must visit every cell; inspection need only visit designated points	19.4, Chapter 16
Route optimization	Finding the shortest tour among the inspection points (the traveling salesman problem)	19.4
Map	Updating the prior map as the	19.6, Chapter 13

maintenance	environment changes, to keep localization possible for the long term	
Multi-session SLAM	Merging trajectories from several runs on different dates into one map	19.6, Chapter 15
Digital twin	The three-dimensional model and asset record built from inspection data	19.7, Chapter 17
Degenerate site	An area of insufficient geometric constraint where localization easily fails (long corridors, empty halls)	19.5, Chapters 17 and 18

19.1 Why Inspection Should Be Autonomous

Inspection is an old and tedious job: someone with a record sheet walks a fixed route once a day, reads instruments, listens to equipment, feels the temperature of pipes, and checks doors, windows, and fire safety equipment. The job looks simple, but it has three problems that human labor cannot overcome.

Three Structural Problems of Manual Inspection

- Consistency: people tire, people are distracted, and people skip what they checked yesterday. When the same inspection point is checked by different people at different times, the standard is hard to keep consistent.
- Frequency: labor is limited, and the inspection frequency is usually pressed to a minimum. An inspection twice a week means a piece of equipment may spend as long as three days in an abnormal state with no one aware of it.
- Environmental risk: substations, high-voltage equipment, underground culverts, buildings after a disaster, radiation areas—some places people should

not be going to often in the first place.

The autonomous inspection robot answers all three directly. Its records are entirely consistent, it can run once a day or even once an hour, and it can enter places people should not. More importantly, it raises inspection from finding faults to predicting them—because only when the data are dense and consistent enough can a trend be seen.

Four Differences between Inspection and Ordinary Navigation

Students readily take inspection to be navigation plus photography, but the engineering emphases of the two are in fact quite different:

Aspect	Ordinary Navigation	Inspection Task
Route	May differ each time; the aim is to arrive	The same every day; the aim is repetition
Localization requirement	Meter level suffices for safe travel	Centimeter level, or the same angle cannot be photographed
Map	Built once per run	Built once and used for a year; must be maintained
Criterion of success	Whether it arrived	Whether the data are comparable

The word repeat is the key. The value of inspection data comes from comparison across time—today's infrared thermal image must be compared with last week's to know whether the temperature is rising. If the position and angle of two shots differ by half a meter, the images cannot be compared directly and all trend analysis loses its foundation. This is the problem Section 19.5 addresses.

19.2 The Technology Stack of an Inspection Robot

An inspection robot is the sum of the first eighteen chapters of this book. This section takes its five layers apart from the bottom up and points out which chapters each corresponds to.

Layer	Content	Chapters of This Book
Sensing layer	3D LiDAR, IMU, wheel encoders, RGB and infrared thermal imaging, microphone	Chapters 7, 8, and 9
Localization and mapping layer	3D SLAM mapping, localization against a prior map, loop closure correction	Chapters 17 and 18
Navigation layer	Global path planning, local obstacle avoidance, control of arrival at inspection points	Chapter 3, Part VIII
Task layer	Inspection point management, route scheduling, battery and recharging decisions	19.4, 19.6
Data layer	Tagging data with position and time, anomaly detection, digital twin and reporting	19.7, Part IX

The Inspection Task Layer: the Layer This Chapter Adds

The first three layers were treated fully in Chapters 17 and 18 and in Part VIII; what this chapter genuinely adds are the task layer and the data layer. The questions the task layer must answer sound administrative but are in fact all optimization problems: in what order should the twelve inspection points be visited? With forty percent of the battery left, return to charge first or finish the round? If positioning at some point failed today, how many retries before giving up?

The quality of these decisions decides directly how much work the robot completes each day. Section 19.4 will show with an experiment that merely putting the inspection points in the right order saves forty-five percent of the distance traveled—and distance traveled is battery, and battery is how many rounds a day can be run.

19.3 Platform Selection: Wheeled, Tracked, Quadruped, and Aerial

There is no universal platform for inspection robots, and the choice is decided almost entirely by the site. This section compares the four mainstream options.

Platform	Advantages	Limitations	Typical Sites
Wheeled	Best endurance, lowest cost, most stable SLAM	Cannot climb stairs, poor over obstacles	Plants, corridors, machine rooms, substations
Tracked	Good over obstacles and ramps, highly stable	Large and heavy, wears the floor when turning	Outdoors, disaster sites, construction sites
Quadruped	Strongest adaptation to terrain, can climb stairs	Short endurance, high cost, gait vibration unkind to SLAM	Multiple floors, culverts, uneven ground
Aerial	Can reach heights and the underside of bridges, fast	Endurance measured in minutes, hard to localize indoors	Bridges, towers, the exterior of large structures

The SLAM Difficulty of a Quadruped Platform

Quadruped robots have become a popular choice for inspection in recent years, but they are a severe test for SLAM. The reason is gait vibration: every footfall produces

high-frequency pitch and roll of the body, and the motion distortion compensation discussed in Section 18.3 is especially prone to error under violent motion of this non-constant-velocity kind. This is also why a quadruped platform almost always requires a high-quality IMU and tightly coupled LiDAR-inertial odometry (Section 18.5).

Three Questions for Choosing a Platform

Does the site have changes in level? Stairs or a threshold rule out a wheeled platform.

How long is one inspection round? Beyond an hour, the endurance of quadruped and aerial platforms becomes the bottleneck.

At what height are the inspection targets? On a ceiling or under a bridge, no ground platform, however capable, can see them.

The last question is the most practical: will SLAM degenerate at this site? A long corridor or an empty hall (Section 17.3, Figure 18-2) is a difficulty for any platform and must be confirmed before purchase.

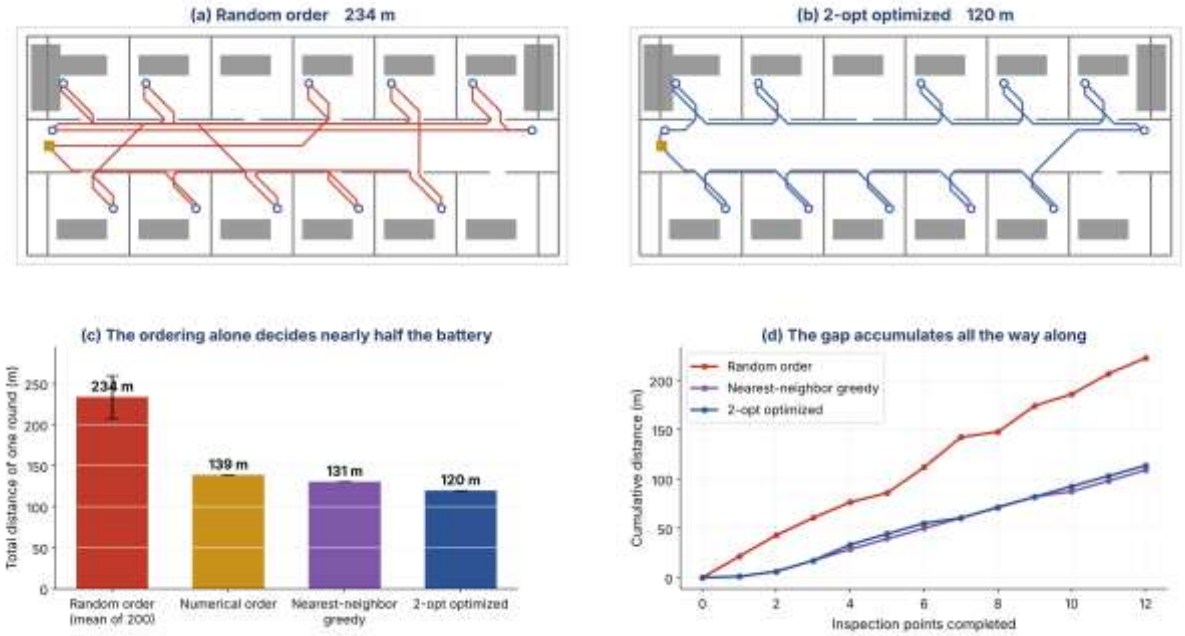
19.4 Inspection Point Planning and Route Optimization

Inspection differs fundamentally from the cleaning task of Chapter 16 in one respect: cleaning must visit every cell (coverage path planning) while inspection need only visit designated points. That difference turns the problem from a coverage problem into a traveling salesman problem—given a set of points that must be visited, find the shortest tour.

The problem is hard in theory (growing factorially with the number of points), but at the practical scale of inspection it is very easy to handle. A floor usually has only a dozen

or so inspection points, and a nearest-neighbor greedy initial solution refined repeatedly with 2-opt gives a near-optimal route within milliseconds.

Figure 19-1 Ordering the Inspection Points: One Route Decides Forty-Five Percent of the Battery



The experiment in this book uses a 34 × 14 meter floor of a department building: one corridor running the length of it, two rows of classrooms and laboratories above and below, an elevator shaft, and a stairwell, with twelve inspection points set along the way—the elevator door, a card reader, a distribution box, a fire hydrant, an air conditioning unit, and laboratory doorways. The robot sets out from the dock at the left end of the corridor, visits all twelve points, and returns to the dock.

Panel (a) is the result of a user tapping the inspection points into an app in no particular order: the path shuttles back and forth along the corridor for a total of 234 meters. Panel (b) is the same twelve points after 2-opt optimization: the robot advances along the corridor in one direction, entering and leaving the rooms on either side as it passes, for a total of 119.7 meters. The points visited are exactly the same and only the order

differs.

Panel (c) sets four orderings side by side. Random order (the mean of 200 trials) is 233.9 meters, numerical order 138.7 meters, nearest-neighbor greedy 130.5 meters, and 2-opt optimized 119.7 meters. From random to optimized saves 49 percent, while going from greedy to 2-opt saves only 8 percent more—which shows something very practical: most of the benefit of ordering comes from not wandering, not from an ingenious algorithm. A ten-line nearest-neighbor method captures ninety percent of the gain.

Panel (d) shows how the gap accumulates. The three curves nearly coincide over the first two or three points, but the random order pays a further price at every additional point, and by the twelfth it has fallen nearly a hundred meters behind. Translated into practical terms: at an inspection speed of 0.5 meters per second, the extra path costs more than three minutes, and if the robot runs eight rounds a day that is 26 minutes more of battery and motor life consumed every day.

Three Corrections in Implementation

The distances in this experiment are the actual shortest traversable paths through the corridor and rooms (computed by BFS on the occupancy grid), not straight-line distances. Ordering by straight-line distance is badly distorted on a floor with partitions.

Real inspection also has ordering constraints: some points must be photographed at a particular time (access control before working hours, say), which makes it a traveling salesman problem with time windows and requires the constraint to be added to the optimization.

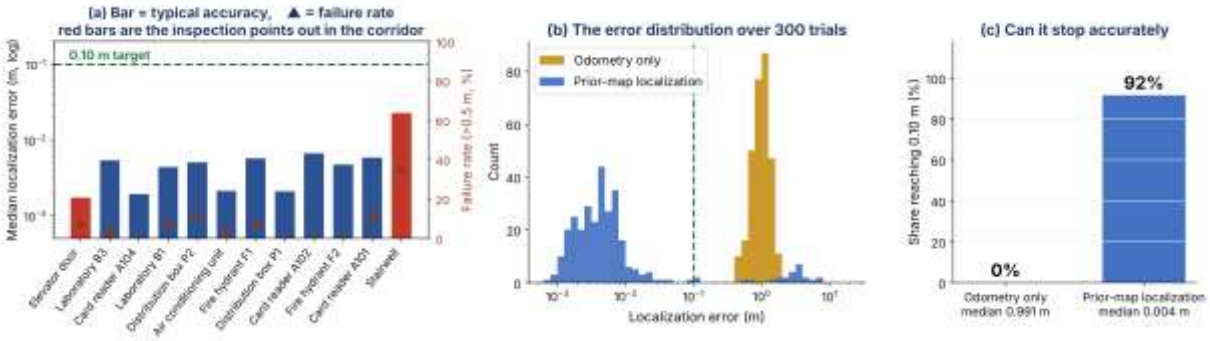
When crossing floors, the elevator is a bottleneck node and the wait is often longer than the walk, so finishing all the points on one floor at once should be set as a hard constraint.

When the battery is not enough for the whole round, the problem becomes maximizing inspection value within a limited distance, and each inspection point then needs a priority.

19.5 Repeat Positioning Accuracy: Can It Stop at the Same Place Every Day

This is the most critical metric of inspection and the one most easily underrated. To photograph the same instrument and compare thermal images of the same wall, the robot must stop at the same position at the same angle every day. The industry requirement is commonly a few centimeters—far above the meter-level requirement of ordinary navigation, which is only not to hit anything.

Figure 19-2 The Same Inspection Point, Repeated 25 Times: Does It Stop Accurately



The experiment is set up as follows. The robot leaves the dock and arrives at an inspection point, having accumulated about 1 meter of odometry error on the way (equivalent to a drift rate of 1.5 percent over a 60-meter route). At this moment there

are two choices—trust the odometry outright, or relocalize by matching a LiDAR scan against the prior map. Each of the twelve inspection points is repeated 25 times, for 300 trials in all.

The two distributions of panel (b) tell the whole story. With odometry alone the error concentrates near 1 meter and not a single trial reaches the 0.10-meter target; with prior-map localization the overwhelming majority of trials fall below 1 centimeter, with a median of 0.004 meters and 92 percent of trials meeting the target. Panel (c) condenses the contrast into one number: 0 percent against 92 percent.

But panel (a) is the real point of this section. It shows the positioning accuracy point by point, and students will notice at once that the blue in-room inspection points have median errors between 1 and 6 millimeters with failure rates of 0 to 8 percent, while the red corridor inspection points—the elevator door and the stairwell—are clearly worse, the stairwell failing 48 percent of the time, with nearly half of all attempts failing to localize.

The reason should by now be very familiar: a long corridor lacks geometric constraint along its axis (the structural degeneracy of Section 17.3, the information matrix analysis of Figure 18-2). The robot knows which wall it is hugging but cannot say how far along the corridor it has traveled. What was still only a theoretical limitation in Chapters 17 and 18 becomes, at an inspection site, a concrete product defect: that inspection point is photographed crooked every other time.

This brings out an important design principle: the position of an inspection point should be decided not only by what is to be checked but also by how accurately the robot can localize there. Moving an inspection point two meters from the middle of the

corridor toward a doorway, so that the robot can see the door frame and the walls on both sides at once, is often more effective than upgrading the sensor—and it costs nothing.

Four Means of Improving Repeat Positioning Accuracy

Move the inspection point: prefer positions rich in geometric features (corners, door frames, places dense with equipment).

Add artificial landmarks: put AprilTags or retroreflective markers (Chapter 4) in degenerate areas to supply an absolute position reference at very low cost.

Complement with multiple sensors: where LiDAR degenerates, as in a corridor, the visual features of a camera (Part VI) or a UWB tag (Chapter 4) can fill in the missing direction.

Accept failure and handle it: when localization confidence is insufficient, have the robot back up a meter and try again, or mark the record as position unreliable, rather than taking a photograph no one knows is crooked.

19.6 Long-Term Operation: Map Maintenance and a Changing Environment

The previous two sections dealt with today's round; this one asks whether it will still run a year from now. This is the greatest difference between inspection and a one-off mapping task: the map is not a deliverable but an asset requiring maintenance.

A real site changes. Desks and chairs are moved, cardboard boxes pile up in the corridor, a laboratory swaps its equipment, a door is open today and closed tomorrow. Each of these changes is small on its own, but together they suffice to defeat localization—

because these are exactly the things scan matching compares.

Figure 19-3 Maps Expire: the Effect of Environmental Change on Localization Success



Panel (a) is the floor on the day it was mapped and panel (b) the same floor some time later—red marks what no longer agrees with the original map: desks and chairs added in the rooms, objects set down temporarily in the corridor, altering about 12 percent of the traversable area in all. From a person's point of view the floor is still the same floor; from the point of view of scan matching, one observation in eight no longer matches.

Panel (c) is the quantified result, and it is the figure in this chapter most worth remembering. The red line is the localization success rate when the old map from the day of mapping continues to be used: 93 percent with no environmental change, falling to 72 percent at 5 percent change and to only 35 percent at 15 percent—in less than a year, half the system is useless. The blue line is the map updated after each inspection round: even at 15 percent environmental change the success rate holds at 74 percent.

The divergence of the two lines shows that map maintenance is not a finishing touch but the watershed of whether an inspection system can operate for the long term. It is worth noting that the blue line is not flat either—it declines gently from 92 percent to 74 percent, because the greater the environmental change, the larger the share of temporary objects (boxes, carts) in the map, and these things will not be there

tomorrow. This is exactly the lifelong mapping problem mentioned in Section 15.8: the map must both remember permanent structure and forget temporary obstacles.

Three Strategies for Map Maintenance

- Periodic rebuilding: rebuild the map at fixed intervals (quarterly, say). The simplest approach, but the system cannot operate during the rebuild and historical correspondences are lost.
- Incremental updating: merge new observations into the map after each inspection round, controlling how fast old information is forgotten by means of upper and lower bounds on the occupancy probability (Section 13.5). The blue line of this experiment is of this kind.
- Layered maps: divide the map into a permanent layer (walls, columns, door frames) and a transient layer (furniture, stored items). Localization uses the permanent layer only while obstacle avoidance uses both. This is the most common practice in commercial systems today and best satisfies remembering and forgetting at once.

19.7 The Functions of Inspection Robots and Their Industrial Applications

The preceding six sections dealt with how to get to a given point. But what an inspection robot does once it arrives, and how well it does it, is the real reason a user pays for it. This section lays out the functions of an inspection robot, then explains how requirements differ across industries by site, and finally introduces the roles that make up this industry chain.

1. The Core Functions of an Inspection Robot

The functions of an inspection robot fall into four classes: moving, collecting, interpreting, and managing. The list below gathers the capabilities that commercial models generally possess or are now introducing, and marks the technology each requires and the corresponding chapters of this book.

Class	Function	Technology Required	Chapters of This Book
Moving	Autonomous navigation and obstacle avoidance	3D SLAM, path planning, cost maps	Chapters 17 and 18, Part VIII
Moving	Docking at a point (centimeter-level repeat positioning)	Localization against a prior map, fine positioning control	19.5, Chapter 11
Moving	Automatic recharging and endurance management	Battery model, scheduling decisions	19.4, 19.6
Moving	Movement between floors (elevator integration)	Topological map, equipment communication interface	19.4, 19.8
Collecting	Visible-light photography at fixed points	Gimbal control, exposure compensation	Chapter 9
Collecting	Infrared thermal measurement	Thermal camera, temperature calibration	Chapter 9, 19.7
Collecting	Capturing abnormal sound and recording spectra	Microphone array, acoustic analysis	19.7

Collecting	Gas, temperature and humidity, and radiation detection	Environmental sensor integration	19.7
Interpreting	Reading instrument needles and digits	Image recognition, OCR	Part IX
Interpreting	Detecting defects in equipment appearance	Object detection, anomaly detection	Part IX
Interpreting	Analyzing hot spots and temperature rise trends	Comparison of data across time	19.5, 19.6
Managing	Tagging data with three-dimensional coordinates and time	Pose recording, time synchronization	Chapter 10, 19.6
Managing	Grading anomalies and dispatching work orders	Rule engine, back-office integration	19.7
Managing	Digital twin and asset records	Three-dimensional reconstruction, point cloud colorization	Sections 17.5 and 17.6

Students can see a clear dividing line in this table: the two classes of moving and collecting are already quite mature in commercial models, and their technical foundations all lie in Parts I through VIII of this book; interpreting is the frontier at present, with most models still only transmitting data back for a person to interpret, while automatic interpretation is gradually being introduced—which is precisely the

subject of Part IX.

2. How Functional Requirements Differ by Site

There is no universal specification for an inspection robot. The functions needed at a power facility and in an underground pipe have almost nothing in common, though both are called inspection. The table below gathers the differences across four common sites, and may serve as a starting point for choosing a project topic or writing a requirements document.

Site	Principal Inspection Targets	Key Functions	Environmental Difficulties
Power facilities	Transformers, switchgear, capacitors, terminals	Infrared thermography, abnormal sound detection, instrument reading	Outdoors in all weather, electromagnetic interference, high-voltage clearance
Bridges and structures	Cracks, corrosion, cable force	High-resolution imagery, three-dimensional positioning, surface orthophotos	No satellite signal, work at height, sub-millimeter resolution required
Plants and warehouses	Piping, machines, fire safety equipment, racking	Imagery at fixed points, temperature, gas detection	People and vehicles mixed, daily environmental change, the line cannot stop
Underground pipes and	Structural deformation,	Waterproofing and dust protection, gas	No light source, confined,

culverts	standing water, gas	detection, lighting	communications cut off, rescue impossible

Bridge inspection best illustrates how function decides platform. To identify cracks of 0.2 to 0.3 millimeters, the vehicle must be able to approach the surface of the structure closely and the camera must have sufficient resolution; and since satellite signals do not reach under a bridge or inside a structure, the vehicle must maintain its position entirely by its own SLAM. These two requirements pull against each other—the closer it flies the harder localization becomes—which is the core engineering difficulty of structural inspection drones and the meeting point of Parts V and VII.

3. Quantified Benefits: How Functions Turn into Value

A purchasing decision for an inspection robot usually looks not at technical metrics but at three quantifiable benefits. The figures below come from cases actually deployed in Taiwan and may serve as a reference for estimation.

- Inspection frequency: after one domestic power utility introduced an outdoor inspection robot, the inspection frequency of the same substation rose from twice a week by staff to once a day by robot. This means the maximum delay before an anomaly is found fell from three days to one, and it gave predictive maintenance a sufficiently dense data foundation for the first time.
- Computing resources: one domestic research institute reports that the navigation algorithm it developed for inspection robots occupies only 25 percent of the robot's computing resources, where traditional algorithms require 65 to 70 percent. For a robot that must run navigation and image recognition at the same time, the computing resources saved decide directly whether it can react in real time.

- Inspection coverage: taking bridges as an example, Taiwan has more than 23,000 road bridges and the competent authority's goal is 100 percent inspection. A quantity of this order makes manual inspection infeasible in principle, and shows that at some sites automation is not a question of efficiency but of feasibility.

4. The Roles That Make Up the Industry Chain

The inspection robot is one of the few robotics applications with a complete supply chain in Taiwan. The following explains the role of each link by industry, so that students may understand the shape of this industry and possible directions for employment.

Link	Industry	Role in the Inspection Robot
Sensing elements	LiDAR manufacturing	Supplies 2D / 3D LiDAR, deciding mapping quality and cost
Sensing elements	Optical and thermal imaging modules	Visible-light cameras, infrared thermal cameras, gimbals
Actuation components	Electrical machinery and motor manufacturing	Wheel sets and joint motors, deciding endurance and the ability to cross obstacles
Control components	Industrial and robot controller industry	Motion control, real-time systems
Computing platform	Edge computing and embedded systems	The onboard computer, deciding how complex an algorithm can be run
Whole-machine integration	Electronics contract manufacturing and systems integration	Mechanical design, whole-machine assembly, mass production and validation

Software and algorithms	Robot software and AI services	SLAM, navigation, image recognition, back-office platform
Research institutes	Industrial technology research institutes	Platform technology development, industry connection, field validation
End users	Utilities, manufacturers, transport authorities	State the requirements, provide the validation site, purchase and deploy

The link worth noting is the research institute. In Taiwan, non-profit research institutes often play the role of platform developer and industry connector: the institute designs the body and the AI core, suppliers in various industries provide components and whole-machine integration, and a utility or manufacturer finally provides the site for validation. This division of labor lets a single company of limited resources take part in highly complex robot development, and it is the model of cooperation students are most likely to encounter should they enter this field later.

From Function to Specification: Writing an Inspection Requirements Document

Define the inspection targets and the criteria of interpretation first: what is to be examined, and to what degree something counts as abnormal. This step decides the requirements for camera resolution, thermal accuracy, and docking accuracy.

Then define the site conditions: whether there are changes in level, whether it is outdoors, whether satellite signals reach it, and how often the environment changes. This step decides the platform type and the map maintenance

strategy.

Then define the frequency and the endurance: how many rounds a day, how long a round takes, and how long charging takes. This step decides the battery capacity and how important route optimization is.

Only then discuss technical specifications. Writing the requirements before choosing the specifications is the most effective way to avoid buying the most expensive sensor and still not solving the problem—an extension of the discussion of sensor selection in Chapter 8.

19.8 A Field Project: the Corridors, Elevators, and Laboratories of a University of Technology in Southern Taiwan

This section turns the preceding seven sections into an executable project plan. The site is a department building at a university of technology in southern Taiwan—a campus of this kind is an excellent site for teaching inspection: it has long corridors (degenerate scenes), elevators and stairs (crossing floors), laboratories (valuable equipment and access control), and genuine management needs.

Site Analysis: the Different Challenges of Three Environments

Site	Principal Challenge	Technical Remedy	Corresponding Chapters
Long corridor	Geometric degeneracy along the axis, high localization failure rate	Tight IMU coupling, artificial landmarks, inspection points moved to doorways	Sections 17.3, 18.5, 19.5

Elevators and stairwells	Crossing floors, a discontinuous map, metal reflections	Layered maps, the elevator as a topological node, a division of labor between quadruped and wheeled	19.4, 19.6
Laboratories	Daily environmental change, people moving about, equipment replaced	Layered maps, dynamic object rejection, incremental updating each round	19.6, Section 13.5
Lobbies and atriums	Open space, sparse returns at a distance, glass curtain walls	Reduce reliance on LiDAR, assist with vision, shorten the spacing between inspection points	Chapter 8, Part VI

A Five-Stage Project Plan

1. Stage 1—data collection: carry a 3D LiDAR and an IMU on a handcart or a robot platform and record datasets in the corridor, the stairwell, and the laboratory separately. This step needs no autonomous navigation—pushing it along by hand will do—and suits an introductory undergraduate assignment.
2. Stage 2—offline mapping and comparison: run LeGO-LOAM, LIO-SAM, and Fast-LIO (Section 18.4) on the recorded data, compare their trajectory accuracy, map quality, and computational burden at each site, and quantify map quality with the wall thickness method of the case in Chapter 17.
3. Stage 3—degenerate scene experiment: focusing on the long corridor, compare pure LiDAR against tightly coupled LiDAR plus IMU and quantify the improvement the IMU brings. This is the part with the most research value, and it corresponds directly to the two tall red bars of Figure 19-2(a) in this chapter.

4. Stage 4—setting inspection points and optimizing the route: set ten to fifteen inspection points on the map you built, implement the nearest-neighbor plus 2-opt planning of Section 19.4 of this chapter, and actually measure the distance traveled and the time taken before and after ordering, reproducing Figure 19-1.
5. Stage 5—long-term operation and map maintenance: run the inspection daily for several consecutive weeks, record the localization success rate at each inspection point, observe how it falls as the environment changes, then introduce incremental updating or a layered map and quantify the improvement. This stage can be extended into a master's topic.

Value for Teaching and Research

The strength of this project is that its levels are distinct: Stages 1 and 2 suit undergraduate coursework, Stages 3 and 4 suit project students, and Stage 5 can be extended into a master's thesis. And the results are concrete and presentable—a three-dimensional model of the campus and a set of daily inspection records are far more persuasive than any slide deck when explaining the work to outsiders.

More importantly, it uses everything in the first eighteen chapters of this book: Chapter 8 to choose the LiDAR, Chapter 7 to pair it with an IMU, Chapter 10 to calibrate the extrinsics, Chapter 17 to process the point cloud, Chapter 18 to run LOAM, and Chapter 12 for loop closure optimization, plus this chapter's route planning, repeat positioning, and map maintenance. A student who has completed these five stages already has the full capability to join an industrial inspection robot team.

Formulas and Procedures

1. Three Evaluation Metrics for an Inspection Task

The quality of an inspection system can be measured by three metrics, which pull against one another:

inspection completion rate = inspection points at which data were collected successfully / total inspection points

repeat positioning accuracy = the standard deviation of the pose over repeated visits to the same inspection point

time per round = total distance traveled / average speed + Σ the collection time at each point

Raising the completion rate usually means more retries and so lengthens the time per round; and shortening the time may sacrifice positioning accuracy. This engineering trade-off is the core of product design, and it is exactly isomorphic with the three metrics of the cleaning robot in Section 16.4.

2. The Route Optimization Problem of Inspection

Let the set of inspection points be $P = \{ p_1, \ldots, p_n \}$, the dock be p_0, and $d(i, j)$ be the length of the shortest traversable path between two points (not the straight-line distance); find a permutation π that minimizes the total distance traveled:

$$\min \; d(p_0, p_{\pi(1)}) + \sum_{k=1}^{n-1} d(p_{\pi(k)}, p_{\pi(k+1)}) + d(p_{\pi(n)}, p_0)$$

This is the traveling salesman problem. The practical solution has two steps: find an initial solution with the nearest-neighbor greedy method, then swap the order of any two segments of the route repeatedly with 2-opt until no further improvement is possible. The experiment of Figure 19-1 in this chapter shows that the greedy method alone already captures more than ninety percent of the benefit.

3. The Occupancy Probability Update of Map Maintenance

The core of incremental updating is to let old information be overwritten by new

observations. Use the log odds of Section 13.2, but set upper and lower bounds on it:

$$l_t = \mathrm{clamp}(l_{t-1} + l_{meas}, -l_{max}, +l_{max})$$

The bound l_{max} decides the plasticity of the map: the larger l_{max}, the more the map trusts history and the harder it is for a new observation to change it; the smaller l_{max}, the more readily the map forgets. A long-term inspection system usually takes a smaller l_{max} in exchange for adaptability to a changing environment.

4. A Day in the Life of an Inspection Robot

Stage	Action
① Startup	Leave the dock and perform global relocalization against the prior map (Chapter 11)
② Planning	Compute this round's route from the inspection point list and the battery level (Section 19.4)
③ Inspection	Travel to each inspection point → position finely to the centimeter → collect imagery, thermal images, and sound
④ Monitoring	Record the localization confidence in real time; retry or mark the data unreliable when confidence is insufficient
⑤ Recharging	Return to the dock once the battery threshold is reached, then resume the points not yet completed
⑥ Updating	Merge this round's scans into the map (incremental updating or a layered map, Section 19.6)
⑦ Reporting	Tag the data with three-dimensional coordinates and time, and output the anomaly list and trend charts (Part IX)

Cases

Case 19-1 From Twice a Week to Once a Day: What a Change in Inspection Frequency Changed

One domestic power utility began research on intelligent inspection robots in 2017 and, working with a non-profit research institute and domestic robot companies, took about three years to complete an outdoor inspection robot whose design and manufacture were entirely in domestic hands, deploying it at a secondary substation.

Its sensor configuration is almost a complete implementation of Part II of this book: a 3D LiDAR builds the surrounding three-dimensional terrain, a 3D camera detects obstacles to avoid collisions, a dual imaging module on a gimbal (including infrared thermography) serves as its eyes, a microphone array captures the sound of the equipment, and an antenna supports remote control. It walks a fixed route, photographs at set points, and returns to its own shed to charge wirelessly when the task is done—precisely the inspection point and repeat positioning requirements described in Sections 19.4 and 19.5.

But the real value of this case lies not in the hardware but in an institutional change: the inspection frequency of the same substation rose from twice a week by staff to once a day by robot. That seemingly ordinary number means the maximum delay before an anomaly is found fell from three days to one, and it gave predictive maintenance a sufficiently dense data foundation for the first time—so that when a transformer terminal begins to degrade, for instance, an outage for replacement can be scheduled before the failure occurs.

The case also presents the limitations of the time honestly: the robot only transmitted imagery back and could not yet identify equipment or interpret sounds by itself, with AI interpretation of instrument needles and detection of abnormal transformer noise

planned for later. This order of solving position first and understanding second is exactly the structure of this book—Parts I through VIII treat where, and only Part IX treats what.

Case 19-2 When There Are Twenty-Three Thousand Things to Inspect

Taiwan has more than 23,000 road bridges, and the competent authority's goal is to inspect them all. The number alone explains why automation is necessary—if manual inspection of each bridge takes several days, one round of national inspection is a job measured in years. This is not a question of efficiency but of feasibility.

The approach developed by domestic research teams splits the problem in two: first digitize the bridge with a drone, building a three-dimensional model and producing an orthophoto of the surface; then let AI find cracks and deterioration in the imagery and locate the defects back onto the three-dimensional model. The system can also detect occluded areas automatically and raise coverage with additional compensating imagery—the same line of thinking as the coverage path planning of Chapter 16, only moved from the ground into the air.

Two technical limitations are worth remembering. The first is positioning: satellite signals do not reach under a bridge or inside a structure, so the drone must maintain its position entirely by its own SLAM, which is exactly where Parts V and VII intersect. The second is resolution: to identify cracks of 0.2 to 0.3 millimeters the drone must fly close enough and the camera must be good enough, and flying close makes positioning harder still. This contradiction is the core engineering problem of structural inspection drones.

Another route is to abandon a general-purpose platform for a dedicated mechanism.

For cable inspection, a domestic engineering consultancy developed a cable-climbing vehicle: suitable for cables up to 300 millimeters in diameter, weighing 13.5 kilograms, climbing at angles above 60 degrees and at speeds above 40 centimeters per second, with sensors mounted at several points on the same cable to measure modes, frequencies, and cable force directly in place of the traditional vibration method. This reminds students that inspection need not use a robot that walks: a dedicated mechanism designed for a single task is often more reliable and cheaper.

Case 19-3 A Domestic Quadruped Platform: Industrial Division of Labor and Simulation First

In July 2026 a domestic competent authority announced the results of a domestic quadruped robot development platform, led by a non-profit research institute, which took about a year and a half to produce two domestic prototypes. The value of this case lies not in any single technology but in the industrial structure it reveals.

The division of labor is quite clear: an electronics contract manufacturer handled whole-machine integration, a vision systems company supplied the imaging payload, an electrical machinery company supplied the joint motors, a LiDAR company supplied the sensing elements, and a controller company handled motion control, while the research institute took charge of the body design, the AI core, the simulation environment, and the inertial measurement unit. In other words, from sensing to actuation and from controller to whole machine, the entire supply chain is domestic, and the platform uses an open architecture so that other teams can develop applications on it.

The applications already validated include underground culvert inspection,

autonomous navigation inspection, swarm cooperation, and firefighting rescue—the first three being the subject of this chapter, while firefighting rescue echoes the discussion of extreme environments in Part VII. The institute expects the first wave of demand in industrial inspection, heavy-load semiconductor transport, and logistics.

What students should note most is the training method: the team used a supercomputer with a general-purpose simulation platform to train more than four thousand virtual robot dogs in parallel simultaneously, then deployed to the physical machines. This simulate-first-then-deploy procedure is the full subject of Chapter 33, and it is already standard practice in a national-level project rather than an experiment in a research laboratory.

Hands-On Activities

Exercise 19-1 Inspection Route Planning and Repeat Positioning Assessment

Purpose: implement in code the two core problems of the inspection task layer—route optimization and repeat positioning assessment—and measure their effects in simulation or at a real site. This exercise is mainly in Python, and Parts One and Two can be completed without a 3D LiDAR.

Part One: Route Optimization (everyone)

1. Take a floor plan (the occupancy grid map produced in the Chapter 13 exercise will serve, or define one yourself on graph paper) and mark 10 to 15 inspection points and a dock position on it.
2. Compute the shortest traversable path length between every pair of points with BFS or A* and build a distance matrix. Note particularly that traversable distance

must be used, not straight-line distance.

3. Implement the nearest-neighbor greedy method to find an initial tour order and compute the total distance.
4. Implement 2-opt refinement: repeatedly try reversing any segment of the route, accept it if the total distance shortens, and stop when no further improvement is possible. Record the improvement.
5. Compare with a random order (the mean of 100 trials), reproduce the four bars of Figure 19-1(c), and compute how much distance ordering saves at your own site.

Part Two: Repeat Positioning Assessment (everyone, or university level)

1. In a simulation environment (or on a real robot), have the robot travel from the same starting point to the same inspection point 20 times, adding different odometry noise each time.
2. Record the final pose error under two conditions: odometry alone, and relocalization by matching a scan against the prior map.
3. Plot the error distributions of the two (a logarithmic horizontal axis is recommended), compute the median and the success rate, and reproduce Figures 19-2(b) and (c).
4. Repeat at a different inspection point, choosing in particular one in the middle of a long corridor and one at a corner of a room, compare their failure rates, and verify the argument of Section 19.5 about site geometry.
5. Adjust the inspection point positions according to the result: move the worst-performing point by one or two meters, measure again, and record the improvement.

Part Three: Long-Term Map Maintenance (university level and project students)

1. Add several new objects at random in the simulation environment to simulate environmental change, and compute the proportional change in traversable area.
2. Run localization with the original map and record the success rate; then run it with the updated map, compare the two, and reproduce Figure 19-3(c).
3. Implement a layered map: mark cells observed on several consecutive rounds as the permanent layer and those appearing on only one or two rounds as the transient layer. Localize using the permanent layer alone and compare its success rate with localization on the full map.
4. (Advanced) Map a real site daily for a week and observe which areas change most often, drawing a heat map of change as a basis for siting inspection points and for a map maintenance strategy.

Discussion Questions

1. At your site, how much distance does 2-opt save relative to a random order? If the robot's battery carries it 2 kilometers, how many more rounds a day does optimized ordering give it?
2. Which inspection point has the worst repeat positioning? What are the geometric features around it? Is this the same thing as the structural degeneracy of Section 17.3 and the information matrix analysis of Figure 18-2?
3. If an inspection point is moved from the middle of the corridor to two meters from a doorway, how much does localization improve? What does this free improvement say, set against the cost of upgrading the sensor?

4. At what proportion of environmental change does the localization success rate of the old map begin to fall appreciably? Using that proportion, how often does your site need its map updated?
5. (University level) Using the platform comparison of Section 19.3, which inspection robot would you choose for your department building? How would the need to cross floors change your choice?

Extended Challenge (university level)

First: implement inspection scheduling with time windows—some inspection points must be visited during a designated period (access control before working hours, air conditioning while it runs)—adding this constraint to the acceptance condition of 2-opt and comparing the route length with and without it. Second: design a localization confidence monitor that computes the smallest eigenvalue of the scan matching information matrix in real time (the method of Figure 18-2), flags an inspection record when it falls below a threshold, and plots the confidence of the whole floor as a heat map to find every degenerate area at the site.

Further Reading

- Reports on the outdoor inspection robot of a domestic power utility. (A complete case of a domestic outdoor inspection robot, with its sensor configuration and the change in inspection frequency.)
- Releases from a domestic non-profit research institute on inspection robots and the domestic quadruped robot development platform. (The industrial ecosystem and division of labor of inspection robots.)
- Reports on domestic bridge inspection drones and AI deterioration recognition

systems. (The automation route and technical limitations of structural inspection.)

- T. Shan and B. Englot, "LeGO-LOAM," IROS 2018; T. Shan et al., "LIO-SAM," IROS 2020. (The two 3D SLAM systems most used on inspection robots, extending Chapter 18.)
- S. Macenski et al., the official documentation of slam_toolbox and Nav2. (Map serialization, pure localization mode, and the navigation stack, the software foundation of an inspection system.)
- D. Applegate et al., The Traveling Salesman Problem: A Computational Study, Princeton Univ. Press. (The authoritative reference on route optimization, covering 2-opt and more advanced methods.)
- Literature on long-term localization and map maintenance (lifelong mapping, multi-session SLAM). (The theoretical foundation of Section 19.6 of this chapter.)

Summary

- Manual inspection has the three structural problems of consistency, frequency, and environmental risk; the autonomous inspection robot answers all three with entirely consistent records, a frequency that can be raised, and the ability to enter dangerous sites, and it raises inspection from finding faults to predicting them.
- The four differences between inspection and ordinary navigation are: the route seeks repetition rather than arrival, localization demands centimeters rather than meters, the map must be maintained rather than discarded after use, and the criterion of success is the comparability of the data.

- The technology stack has five layers—sensing, localization and mapping, navigation, task, and data; the first three were treated in Chapters 17 and 18 and in Part VIII, and what this chapter adds are the task layer (inspection points and routes) and the data layer (digital twins and reports).
- Platform selection is decided by the site: wheeled is best in endurance but cannot climb stairs, tracked is good over obstacles, quadruped adapts best to terrain but has short endurance and a gait vibration unkind to SLAM, and aerial can reach heights but has endurance measured in minutes.
- Inspection is a traveling salesman problem, not a coverage problem. The experiment shows 234 meters for a random order, 130 meters for nearest-neighbor greedy, and 119.7 meters for 2-opt optimization, so ordering saves 49 percent, nine tenths of which comes from the simple greedy method.
- Repeat positioning accuracy is the most critical metric of inspection. Odometry alone cannot reach the 0.10-meter target (0 percent), while localization against a prior map reaches 92 percent with a median of 0.004 meters; but inspection points in a corridor can fail 48 percent of the time because of structural degeneracy—the position of an inspection point should take account both of what is to be checked and of how accurately the robot can localize there.
- The map is an asset requiring maintenance. Continuing to use the old map, the localization success rate falls from 93 percent to 35 percent as the environment changes; updating the map every round holds it above 74 percent. The three maintenance strategies are periodic rebuilding, incremental updating, and layered maps.
- The functions of an inspection robot fall into the four classes of moving, collecting, interpreting, and managing; moving and collecting are already quite

mature, interpreting still mainly means transmitting data back for a person to interpret, and automatic interpretation is the frontier at present (Part IX).

- Functional requirements differ enormously by site: power facilities emphasize thermal imaging and abnormal sound, bridge structures need sub-millimeter resolution and positioning without satellites, plants and warehouses face daily change, and underground pipes require waterproofing, dust protection, and gas detection.
- Taiwan's inspection robots have a complete supply chain, made up of LiDAR, optics, motors, controllers, edge computing, whole-machine integration, software algorithms, research institutes, and end users together; deployed cases show that the inspection frequency can be raised from twice a week to once a day.

Carrying Forward

Part V is complete. From the point cloud processing of Chapter 17 and the LOAM and feature registration of Chapter 18 to this chapter's placing of those techniques into an inspection system that must run every day for a year, students should be able to see a clear thread: the algorithm sets the ceiling of a system, while the mundane decisions of the task layer decide how much of that ceiling is actually reached.

The two unusually tall red bars of Figure 19-2(a) in this chapter also point once more to the limit of LiDAR—in a geometrically degenerate long corridor, no 3D LiDAR, however good, can say how far it has traveled. In the next part we change sensors: from Chapter 20 onward we see with a camera. Visual SLAM has difficulties of its own (white walls, repetitive texture, scale ambiguity), but it serves exactly where LiDAR is helpless—the

posters on the walls, the door plates, and the floor texture are all clear features to a camera.